

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Best Bike Paths (BBP) Design Document (DD)

Software Engineering II

Advisor: Prof.ssa. Elisabetta Di Nitto
Academic Year: 2025-2026
Students:

- Giuseppe
- Lorenzo
- Giacomo

Deliverable:	DD
Title:	Design Document
Authors:	Giuseppe, Lorenzo, Giacomo
Version:	1.0
Date:	2026-01-05
Download page:	https://github.com/giusedmb/best-bike-paths-se2
Copyright:	Copyright © 2026 – All Rights Reserved

Contents

Contents **i**

List of Figures **iv**

List of Tables **vi**

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1
1.3 Definitions, Acronyms, and Abbreviations . . . 2
1.4 Revision History . . . 2
1.5 Reference Documents . . . 2
1.6 Document Structure . . . 2

2 Architectural Design **4**
2.1 Overview . . . 4
2.2 Component View . . . 6
2.2.1 High-Level Components . . . 6
2.2.2 Low-Level Components . . . 9
2.3 Deployment View . . . 12
2.4 Runtime View . . . 13
2.4.1 Local Trip Recording and Monitoring . . . 13
2.4.2 Trip Upload and Synchronization . . . 15
2.4.3 Backend Data Processing Pipeline . . . 16
2.4.4 Trip Search and Visualization . . . 17
2.4.5 Manual Obstacle Reporting . . . 19
2.4.6 User Authentication . . . 20
2.4.7 User Registration . . . 21
2.4.8 Manual Trip Creation . . . 22

2.5 Component Interfaces . . . 23
2.5.1 Data Transfer Objects (DTOs) . . . 30
2.6 Selected Architectural Styles and Patterns . . . 39
2.6.1 Microservices Architecture . . . 39
2.6.2 Command Query Responsibility Segregation (CQRS) . . . 40
2.6.3 API Gateway Pattern . . . 40
2.6.4 Polyglot Persistence . . . 40
2.6.5 Containerization and Orchestration . . . 41
2.7 Security and Data Privacy . . . 41
2.7.1 Network and Communication Security . . . 41
2.7.2 Authentication and Identity Management . . . 42
2.7.3 Data Privacy and GDPR Compliance . . . 42
2.7.4 Infrastructure Security . . . 42
2.8 Other Design Decisions . . . 43
2.8.1 Asynchronous Processing and Eventual Consistency . . . 43
2.8.2 Dedicated External Service Adapters . . . 43
2.8.3 Client-Side Data Persistence and Resilience . . . 43
2.8.4 Modular Mobile Application Architecture . . . 44

3 User Interface Design 45
3.1 User Flow Diagram . . . 46
3.1.1 Login . . . 47
3.1.2 User Profile . . . 49
3.1.3 Homepage . . . 51
3.1.4 Auto Recording Trip Interface . . . 53
3.1.5 Trip Completion and Verification . . . 55
3.1.6 Manual Reporting Interface . . . 57
3.1.7 Trip Details and Publication . . . 58
3.1.8 Edit Trip Interface . . . 60
3.1.9 Trip Publication Confirmation . . . 61
3.1.10 Publication Confirmation . . . 62
3.1.11 Map and Navigation Interface . . . 63
3.1.12 Guest Access Interface . . . 65
3.1.13 Conclusions . . . 66

4 Requirements Traceability 67

5 Implementation, Integration and Testing 71

5.1 Overview and Implementation Plan 71
5.2 Features Identification 72
5.2.1 Mapping Features to Architectural Components 73
5.3 Integration Strategy 75
5.3.1 Step 0: Data Access & External Connectors 75
5.3.2 Step 1: User Management Foundation 75
5.3.3 Step 2: Core Processing Algorithms 76
5.3.4 Step 3: Trip and Reporting Orchestration 77
5.3.5 Step 4: Search and Visualization Layer 77
5.3.6 Step 5: Client Application Integration 78
5.4 System Testing Strategy 78

6 Effort Spent **80**

7 References **81**
7.1 References 81
7.2 Used Tools 81

List of Figures

2.1 BBP Architecture Overview 4
2.2 High-Level Components 6
2.3 Low-Level Components 9
2.4 Trip Aggregation Service Internal Architecture 11
2.5 Deployment Diagram 12
2.6 Runtime View - Local Trip Recording 13
2.7 Runtime View - Trip Upload and Synchronization 15
2.8 Runtime View - Backend Data Processing Pipeline 16
2.9 Runtime View - Trip Search and Visualization 17
2.10 Runtime View - Manual Obstacle Reporting 19
2.11 Runtime View - User Authentication 20
2.12 Runtime View - User Registration 21
2.13 Runtime View - Manual Trip Creation 22

3.1 Unified User Flow Diagram for the BBP Mobile Application 46
3.2 Login Page - ST 47
3.3 Signup Page - ST 47
3.4 Mockup of the BBP User Profile UI 49
3.5 Mockup of the BBP Homepage UI 51
3.6 BBP Auto Recording Trip UI 53
3.7 Summary and Anomaly Verification UI 55
3.8 Mockup of the BBP Manual Reporting UI 57
3.9 Mockup of the BBP Trip Details and Publication UI 58
3.10 Mockup of the BBP Edit Trip UI 60
3.11 Mockup of the BBP Trip Publication Name Dialog 61
3.12 Success Feedback 62
3.13 Permanent Published State 62
3.14 Trip Publication Workflow Results 62
3.15 Map Visualization 63
3.16 Path Selection List 63

3.17 BBP Map Navigation and Path Selection UI 63
3.18 Mockup of the BBP Guest Access UI 65

5.1 Integration Strategy - Step 0: Data Access & Connectors 75
5.2 Integration Strategy - Step 1: Users Account Manager Service 76
5.3 Integration Strategy - Step 2: Core Algorithms 76
5.4 Integration Strategy - Step 3: Trip Ingestion and Report Services 77
5.5 Integration Strategy - Step 4: Search and Dashboard 77
5.6 Integration Strategy - Step 5: Client Integration 78

List of Tables

6.1 Effort Spent by Each Member of the Group Table 80

1 Introduction

1.1. Purpose

The purpose of this Design Document (DD) is to provide a detailed and structured description of the architectural and design choices adopted for the Best Bike Paths (BBP) system. The document acts as a technical blueprint that guides the development team through the implementation phase, clarifying how the system requirements identified in the RASD are concretely addressed.

This document aims to ensure a shared and precise understanding of the system's technical solution among all stakeholders involved in the project. By explicitly describing components, interactions, and design rationales, it helps align expectations, reduce ambiguities, and support coordinated development activities.

Furthermore, the DD motivates and documents the main architectural decisions, explaining the selection of technologies, architectural styles, and design patterns. This information is intended to preserve design consistency over time and to serve as a reference for future maintenance, evolution, or extension of the BBP system.

1.2. Scope

As described in the Requirements Analysis and Specification Document (RASD), Best Bike Paths (BBP) is a software system designed to support cyclists in discovering, recording, and evaluating bike paths based on their quality, safety, and overall suitability. The system enables users to record cycling trips, report obstacles and street conditions, and access aggregated information about routes shared by the community.

BBP integrates data collected manually by users and automatically through mobile device sensors, enriching it with external information such as maps and weather conditions. The system processes these inputs to compute scores and consolidated evaluations for bike paths, supporting informed route selection for both registered and unregistered users.

The scope of this Design Document is limited to the technical realization of the BBP system. It focuses on software architecture, component design, data management, interaction flows, and deployment aspects, without redefining functional or non-functional requirements already specified in the RASD.

1.3. Definitions, Acronyms, and Abbreviations

This section reports the definitions, acronyms, and abbreviations used throughout the document. The terminology is consistent with the RASD to ensure traceability and conceptual coherence between requirements and design artifacts.

1.4. Revision History

- **Version 1.0** (2025-01-05): Initial release of the Design Document.

1.5. Reference Documents

- Best Bike Paths (BBP) Requirements Analysis and Specification Document (RASD).
- R&DD Assignment description, Academic Year 2025–2026.
- Provided course materials and lecture notes.

1.6. Document Structure

1. **Introduction**: Presents the purpose, scope, reference documents, and overall organization of the Design Document, providing the necessary context for understanding the subsequent sections.
2. **Architectural Design**: Describes the high-level architecture of BBP, including major components, their responsibilities, interactions, and deployment environment. Architectural styles and key design decisions are discussed and justified.
3. **Detailed Design**: Provides a deeper view of the internal structure of the system, including component interfaces, data models, and main design patterns adopted to satisfy the requirements.
4. **User Interface Design**: Illustrates the design of the user interfaces, describing how cyclists interact with the system through mobile and, where applicable, web interfaces.

5. **Requirements Traceability**: Maps the requirements defined in the RASD to the architectural and design elements described in this document, ensuring completeness and consistency.
6. **Implementation, Integration, and Test Plan**: Outlines the planned development process, integration strategy, and testing activities required to validate the system.
7. **Effort Spent**: Reports the workload distribution and effort spent by each member of the project team.
8. **References**: Collects all bibliographic and technical references cited throughout the document.

2 | Architectural Design

2.1. Overview

The following is an overview of the BBP architecture general components. A more detailed view will be provided in 2.3.

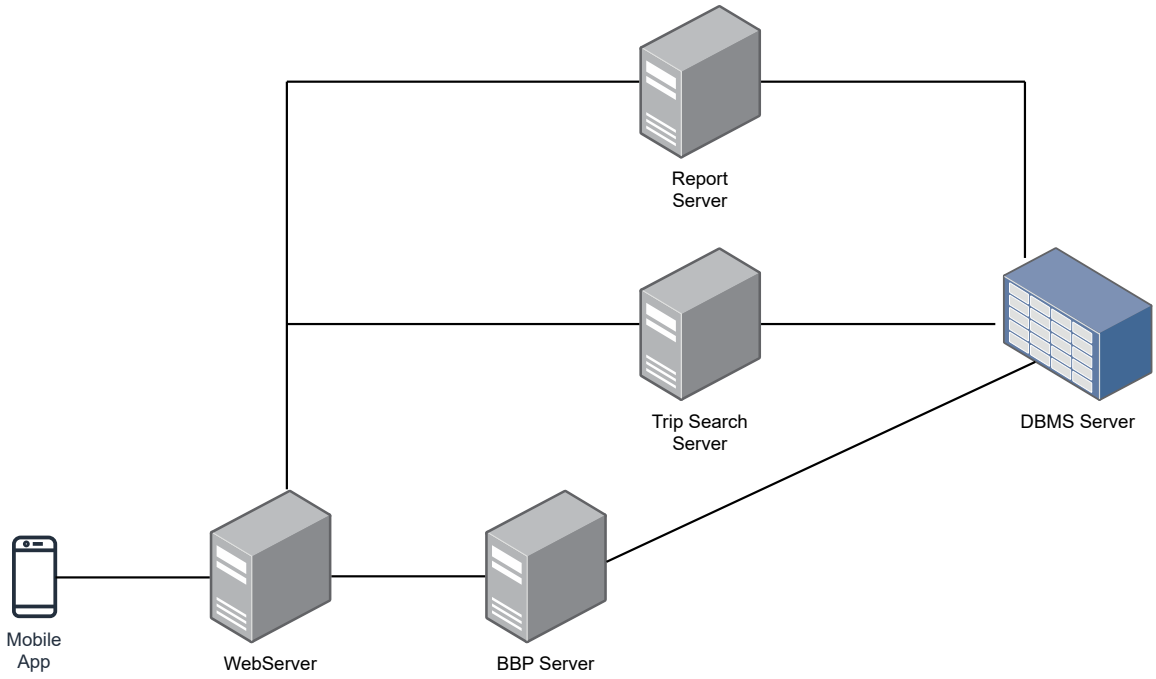


Figure 2.1: BBP Architecture Overview

On the client side can be seen:

- **Mobile App**: The application used by end users to record trips, either manually or automatically. The mobile app is responsible for accessing device sensors (e.g., GPS and motion sensors) and external services such as weather and map providers. Collected and enriched trip data is then sent to the backend for validation, storage, and aggregation.

On the server side, we can see:

- **Web Server**: It handles HTTP/HTTPS communication between users and the BBP system. It acts as an API Gateway, serving the user interface and forwarding API requests to the core logical component, the BBP Server.
- **Trip Search Server**: A specialized server dedicated exclusively to handling path search operations. By receiving read-only requests directly from the Web Server and querying the DBMS, it offloads the main BBP Server, ensuring fast response times even during high traffic.
- **Report Server**: A dedicated server designed to handle crowd-sourced data submissions, such as manual obstacle reports. It operates independently from the high-bandwidth trip ingestion pipeline to process, validate, and persist discrete user feedback, ensuring that spikes in reporting activity do not interfere with real-time tracking performance.
- **BBP Server**: It represents the core backend of the system. It contains the main business logic and orchestrates the various sub-components dedicated to data and trip management. Internally, the logic is divided into the following functional modules:
  - **Trip Search Service**: A specialized module dedicated to read operations. It receives search queries and efficiently queries the DBMS to retrieve paths and information.
  - **Trip Ingestion Service**: The entry point of the writing pipeline. It is responsible for receiving and pre-processing raw telemetry data sent by user devices.
  - **Trip Validation Service**: A processing component that analyzes data received from the Ingestion Service. Its task is to filter GPS errors, verify data consistency, and discard invalid anomalies before proceeding.
  - **Trip Aggregation Service**: A critical component that receives validated data. It is responsible for merging the new trip information with the existing history for that road segment, updating the collective status of the path.
  - **Trip Scoring Service**: The final step in the processing chain. It calculates or updates the quality "Score" of the path based on aggregated data and persists the final results into the DBMS.

2.2. Component View

2.2.1. High-Level Components

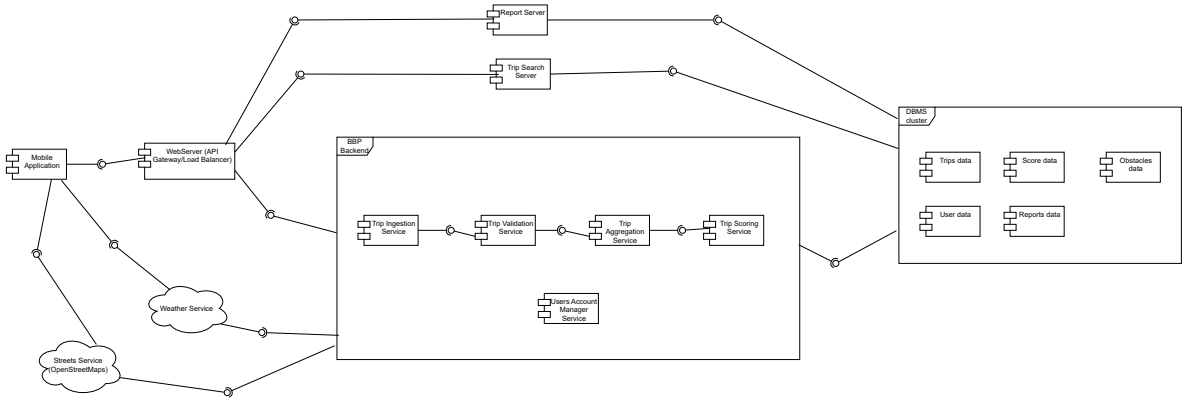


Figure 2.2: High-Level Components

Figure 2.2 shows the system architecture. It adopts a **Microservices** pattern to ensure modularity, scalability, and high availability. The components are organized into logical layers: Client, Entry Point, Backend Services, Data Persistence, and External Systems.

Client Side

Mobile Application. The mobile application represents the primary interface for the end-user (Cyclist). It is responsible for acquiring high-frequency telemetry data, including GPS, accelerometer, and gyroscope measurements, directly from the device sensors. The application manages user interaction, map visualization, and initiates secure HTTPS requests toward the backend services. To preserve battery life and reduce computational overhead on the client device, it avoids heavy data processing, delegating these tasks entirely to the server-side infrastructure.

Entry Point

WebServer (API Gateway / Load Balancer). The WebServer acts as the single entry point for the *BBP Backend Cluster*, effectively decoupling the client from the internal microservices architecture. Its responsibilities include routing incoming requests to the appropriate backend service, such as forwarding path search requests to the *Trip Search Service* or authentication requests to the *Users Account Manager*. In addition, it performs load balancing by distributing incoming traffic across multiple active service instances to

handle peak loads, and it manages security concerns by handling SSL termination and protecting internal endpoints from direct external access.

Backend Services (BBP Backend Cluster)

The backend is composed of independent microservices, each implementing a specific portion of the business logic and data processing pipeline.

Users Account Manager Service. This service manages all logic related to user identity, including registration, authentication, and profile updates. It interacts exclusively with the *User Data* storage to guarantee security and strict isolation of Personally Identifiable Information (PII).

Trip Ingestion Service. The Trip Ingestion Service is a write-intensive component designed to receive raw telemetry streams from the mobile application. It acts as a high-throughput buffer to prevent data loss during periods of peak usage. Additionally, it interacts with the external *Weather Service* to enrich incoming trip data with meteorological conditions at the time of recording.

Trip Validation Service. This service is responsible for data quality assurance. It processes raw trip data to filter GPS inaccuracies and validate potential obstacles. To ensure spatial correctness, it queries the *Streets Service (OpenStreetMap)* to perform map-matching operations, verifying that recorded coordinates correspond to actual physical roads.

Trip Aggregation Service. The Trip Aggregation Service implements the collaborative logic of the system. It merges newly validated trip data with the historical state of bike paths and applies majority voting algorithms to resolve conflicting user reports, such as determining the effective roughness of a road segment.

Trip Scoring Service. This service acts as the computation engine responsible for calculating the qualitative *Score* of a path. It processes aggregated information, including surface smoothness, detected obstacles, and user ratings, to generate a numerical value representing the overall safety and quality of the route.

Specialized Independent Servers

To optimize resource allocation and ensure high availability, specific functionalities are hosted on dedicated server infrastructure, physically or logically separated from the core processing cluster. This architectural decision minimizes resource contention and isolates critical read/write workloads.

Trip Search Server. Designed as a read-optimized component, this server implements the query side of the system's architecture (following a CQRS-like pattern). It is dedicated exclusively to handling user pathfinding queries by retrieving pre-calculated scores and obstacle data. By isolating this read-heavy workload on a dedicated server, the system ensures low-latency responses (under two seconds) regardless of the load on the write-intensive ingestion pipeline.

Report Server. This server operates as an autonomous unit dedicated to handling crowd-sourced data submissions, such as manual obstacle reports or generic street feedback. Unlike the continuous telemetry stream processed by the Ingestion Service, user reports are discrete, asynchronous events. This server runs its own validation logic to filter spam or incorrect data before persisting validated reports into the central database, ensuring that spikes in user reporting do not interfere with the real-time tracking performance.

Data Persistence Layer (DBMS Cluster)

To support the heterogeneous requirements of the backend services, the data persistence layer is logically partitioned into a cluster of specialized data stores. The *User Data* store maintains user profiles, credentials, and configuration settings, and is isolated to comply with privacy regulations such as GDPR. The *Trips Data* store is optimized for time-series data and retains the large volume of raw telemetry streams collected from sensors. The *Obstacles & Reports Data* store contains validated obstacles, such as potholes and interruptions, as well as user-submitted reports. Finally, the *Score Data* store holds the computed road quality scores and path statuses, and is optimized for fast spatial queries to efficiently support the *Trip Search Service*.

External Systems

The architecture integrates with external systems to enhance the quality and reliability of the collected data. The *Weather Service* is queried to enrich trip records with real-time meteorological context, such as rain and wind conditions. Additionally, the *Streets Service (OpenStreetMap)* provides authoritative geospatial reference data used for coordinate validation and map-matching against existing road infrastructure.

2.2.2. Low-Level Components

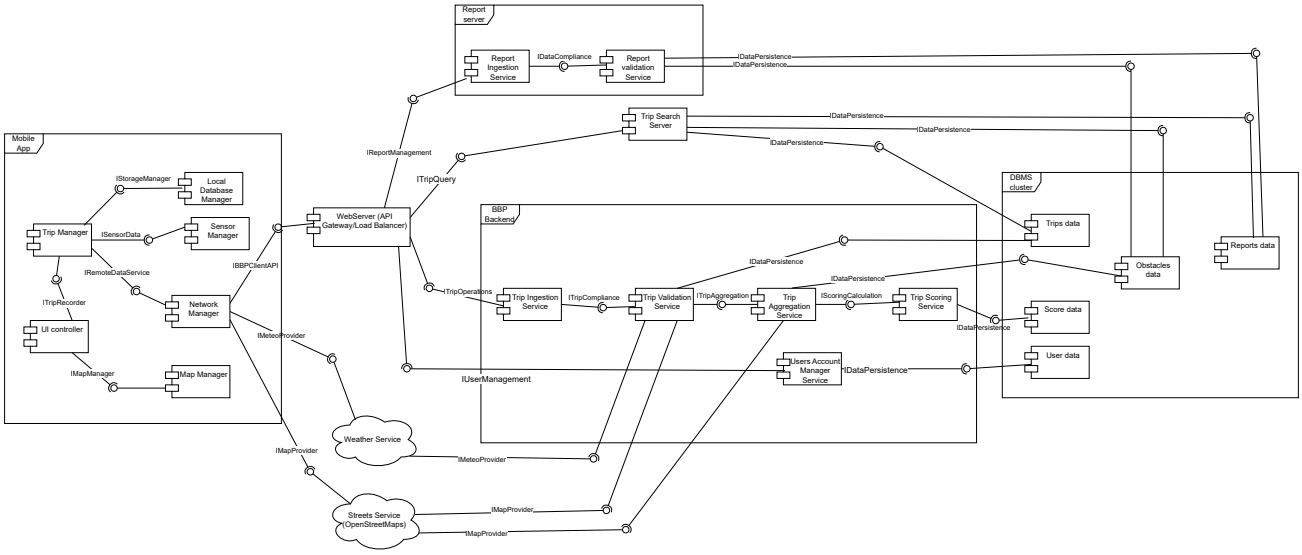


Figure 2.3: Low-Level Components

Mobile Application Internal Components

To ensure responsiveness, fault tolerance, and efficient resource usage, the Mobile Application is architected into distinct functional modules:

UI Controller. This component manages the presentation layer of the application. It handles user interactions (e.g., starting a trip, submitting a report) and updates the visual interface based on the application state. It acts as the bridge between the user and the underlying logic components.

Sensor Manager. A hardware abstraction layer responsible for interfacing with the device's physical sensors (GPS, Accelerometer, Gyroscope). It configures the sampling rate to balance data precision with battery consumption and provides a normalized stream of raw telemetry data to the Trip Manager.

Trip Manager. This component acts as the central orchestrator for the tracking session. It manages the lifecycle of a trip (Start, Pause, Stop) and coordinates the flow of data. It receives raw readings from the Sensor Manager and directs them to the storage layer, ensuring no data is lost even if the network is unavailable.

Local Database Manager. This component ensures data persistence and fault tolerance. It manages a local embedded database (e.g., SQLite) acting as a buffer. During a trip, data is written here first; this guarantees that if the application crashes or network

connectivity is lost, the recorded telemetry is preserved and can be synchronized later.

Network Manager. Responsible for all asynchronous communication with the backend services. It retrieves pending data from the Local Database Manager and handles the upload process to the *Trip Ingestion Service* or *Report Server*. It implements retry policies and data synchronization logic to handle intermittent network conditions transparently.

Map Manager. This component wraps the mapping SDK (e.g., Google Maps or OSMdroid). It is responsible for rendering the map interface, drawing the user's current route, and visualizing map markers such as obstacles or Points of Interest retrieved from the backend.

Report Server Internal Components

To handle crowd-sourced feedback efficiently, the Report Server is internally divided into ingestion and validation responsibilities:

Report Ingestion Service. This module exposes the REST API endpoints required to receive user-generated reports (e.g., manual obstacle signaling). It handles the initial request parsing, payload extraction (including images or text), and acts as the entry point for the reporting subsystem.

Report Validation Service. This component implements the specific business logic to verify user reports before they are permanently accepted. It performs checks such as duplicate detection and spatial verification (querying the *Streets Service* to ensure the reported coordinates match a valid road segment) before persisting the valid report into the *Reports Data* storage.

Note on Auxiliary Services To maintain clarity and focus on the core business logic components (Trip Recording, Processing, and Reporting), auxiliary architectural elements such as the **Notification Service**, **Logging Service**, and **Monitoring Systems** are abstracted from the component diagram. Specifically, a *Notification Service* is envisaged within the backend cluster to handle asynchronous user feedback (e.g., notifying a user when their report has been validated). This service would interact with the *Web Server* (Gateway) to push updates to the Mobile Application using standard protocols (e.g., WebSockets or integration with external push notification providers like FCM/APNS), without altering the primary data flow described in this section.

Trip Aggregation Service Internal Components

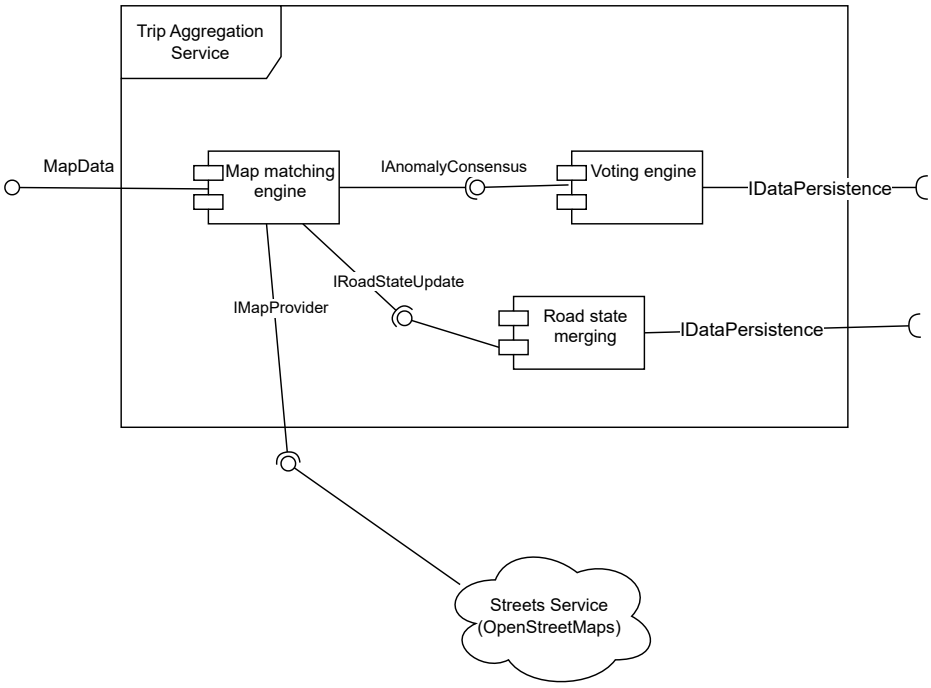


Figure 2.4: Trip Aggregation Service Internal Architecture

The Trip Aggregation Service represents the core decision-making unit of the backend pipeline. To transform noisy telemetry streams into authoritative road data, it is internally decomposed into three specialized sub-components:

- **Map Matching Engine**: This component acts as the entry point for validated trip data. It is responsible for aligning raw GPS coordinates to the actual road network topology. It interacts with the external *Streets Service* (OpenStreetMap) to retrieve road segment metadata (Edge IDs), ensuring that spatial data from different users is correctly associated with the corresponding physical street segments.
- **Voting Engine**: This component implements the consensus logic for discrete anomalies. It receives map-matched anomaly reports (e.g., potential potholes) and applies a majority voting algorithm. An anomaly is confirmed as a verified obstacle only if it is reported by a significant number of distinct users within a specific time and space window, filtering out false positives.
- **Road State Merging**: This component processes continuous road metrics, such as average speed and surface roughness. It employs weighted moving average algo-

rithms to merge new telemetry data with the historical state of the road segments stored in the database. This ensures that the global view of the road conditions evolves smoothly over time, mitigating the impact of outlier measurements.

2.3. Deployment View

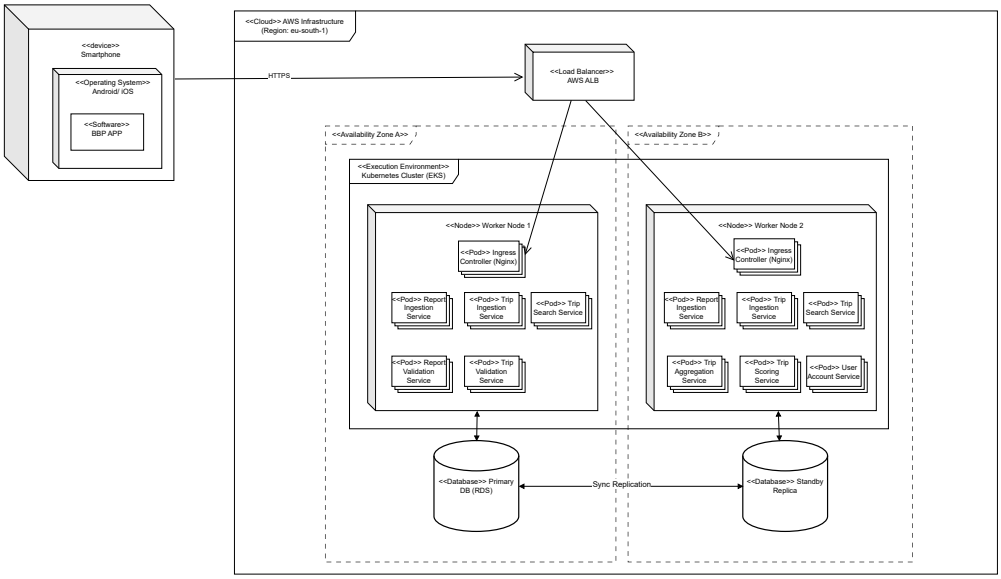


Figure 2.5: Deployment Diagram

Detailed Cluster Configuration

Within the Kubernetes environment, specific resource objects are utilized to manage networking and configuration:

- **Ingress Controller (Nginx):** Handles internal routing of HTTPS requests. It maps external URLs (e.g., `/api/trips`) to the appropriate internal services.
- **Kubernetes Services:** Provide stable internal IP addresses and DNS names (Service Discovery), allowing pods to communicate (e.g., *Aggregation Service* calling *Streets Service*) regardless of the ephemeral nature of individual pod instances.
- **Secrets & ConfigMaps:** Decouple configuration from code. Database credentials and external API keys (e.g., OpenWeatherMap key) are stored as *Secrets* and injected securely into the containers at runtime.

Storage Technologies

The architecture employs a **Polyglot Persistence** strategy tailored to the data nature:

- **AWS RDS (PostgreSQL + PostGIS):** Used for structured business data (Users, Reports) and geospatial queries (finding obstacles within a radius).
- **AWS Timestream (or InfluxDB):** A specialized Time-Series database used to store the high-volume, append-only stream of raw sensor telemetry (accelerometer/GPS traces) for historical analysis.

2.4. Runtime View

2.4.1. Local Trip Recording and Monitoring

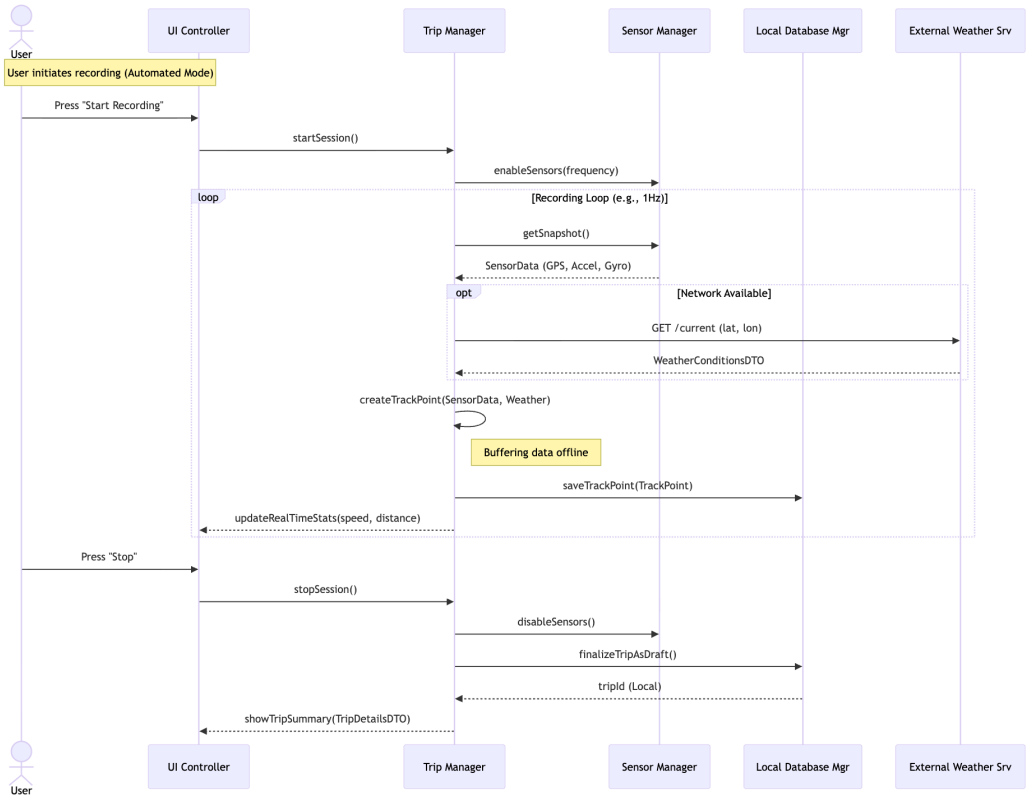


Figure 2.6: Runtime View - Local Trip Recording

Figure 2.6 illustrates the data acquisition loop on the mobile device. This process is designed to ensure robust sensor data collection and real-time user feedback, while maintaining full functionality even in offline conditions.

- **Initialization:** The process begins when the user triggers the "Start Recording"

action. The *UI Controller* invokes the *Trip Manager*, which initializes a new session and activates the hardware sensors via the *Sensor Manager*.

- **Acquisition Loop:** During the ride, the *Trip Manager* periodically polls the *Sensor Manager* to retrieve a snapshot of the telemetry data (GPS coordinates, accelerometer, gyroscope). These raw readings are encapsulated into a `TrackPoint` object. Optionally, if network connectivity is available, the system attempts to fetch real-time context from the *Weather Service* to enrich the user display.
- **Local Buffering:** To guarantee data integrity (Constraint C05), each `TrackPoint` is immediately persisted to the internal storage via the *Local Database Manager*. This ensures that no data is lost in the event of an application crash or battery failure.
- **Finalization:** When the user stops the recording, the *Trip Manager* consolidates the buffered data into a `TripUploadDTO` structure (setting the status to DRAFT) and finalizes the record in the local database. The trip is now ready for review or deferred upload.

2.4.2. Trip Upload and Synchronization

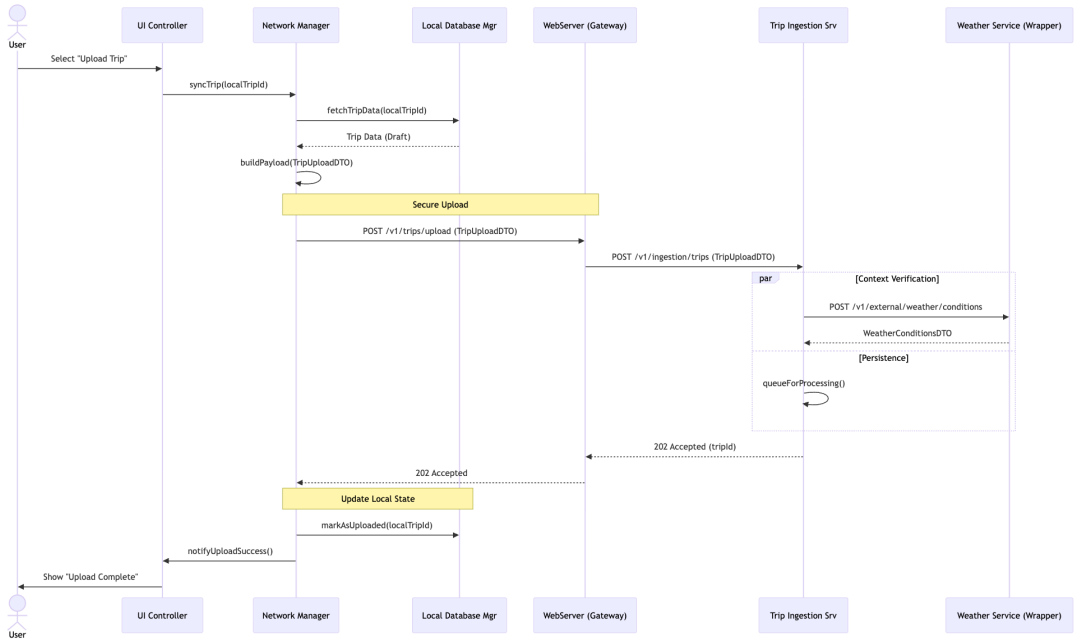


Figure 2.7: Runtime View - Trip Upload and Synchronization

Figure 2.7 details the synchronization workflow. This phase ensures that locally stored trip data is securely transmitted to the backend for processing.

- **Upload Initiation:** The process is triggered by the user (or automatically upon network recovery). The *Network Manager* first interacts with the *Local Database Manager* to retrieve the complete trip payload, encapsulated in a `TripUploadDTO`.
- **Data Ingestion:** The *Network Manager* establishes a secure HTTPS connection with the *WebServer* (API Gateway) and invokes the `POST /v1/trips/upload` endpoint. The Gateway routes this request to the internal *Trip Ingestion Service* via `POST /v1/ingestion/trips`.
- **Verification and Queuing:** Upon receipt, the *Trip Ingestion Service* performs a preliminary check by querying the *Weather Service* (via `POST /v1/external/weather/conditions`) to validate the environmental metadata. It then persists the raw data in the *Trips Data* store and places the task in a processing queue.
- **Asynchronous Acknowledgment:** To maximize responsiveness, the server im-

mediately returns a `202 Accepted` status code once the data is safely queued. The *Network Manager* receives this confirmation and instructs the *Local Database Manager* to mark the trip as "Uploaded", finalizing the synchronization on the client side.

2.4.3. Backend Data Processing Pipeline

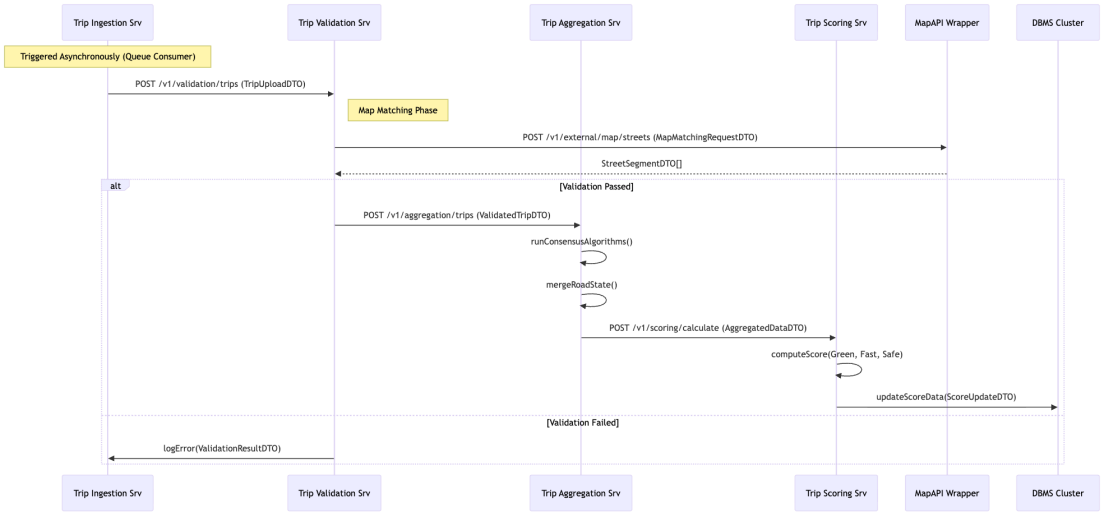


Figure 2.8: Runtime View - Backend Data Processing Pipeline

Figure 2.8 illustrates the asynchronous processing workflow responsible for transforming raw telemetry into meaningful safety scores. This pipeline is triggered automatically by the *Trip Ingestion Service* upon successful data persistence and focuses on the internal orchestration between microservices.

- **Validation:** The process begins when the *Trip Ingestion Service* dequeues the raw trip and invokes the *Trip Validation Service* via `POST /v1/validation/trips`. This component verifies the data consistency and filters GPS noise. Crucially, it performs map-matching by querying the external *Streets Service* (via `POST /v1/external/map/streets`) to ensure the recorded points correspond to valid road segments.
- **Aggregation:** Once validated, the data is forwarded to the *Trip Aggregation Service*

using the `ITripAggregation` interface. Here, the service merges the new trajectory with the historical state of the road segments. This involves running consensus algorithms (via the internal *Voting Engine*) to confirm any reported anomalies.

- **Scoring:** The updated segment data is then sent to the *Trip Scoring Service* via `POST /v1/scoring/calculate`. This engine recalculates the Quality of Experience (QoE) score for the affected paths based on the new averages (roughness, speed, safety).
- **Final Persistence:** Finally, the new scores are persisted in the *Score Data* store via the `IDataPersistence` interface, making them immediately available for future search queries.

2.4.4. Trip Search and Visualization

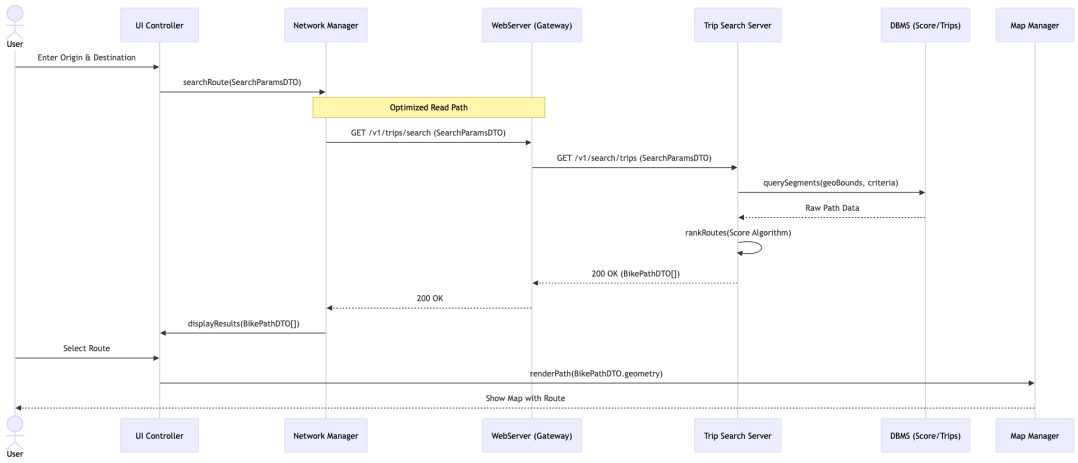


Figure 2.9: Runtime View - Trip Search and Visualization

Figure 2.9 depicts the execution flow for the Route Search scenario. This workflow demonstrates the implementation of the *Command Query Responsibility Segregation* (CQRS) pattern, where read operations are decoupled from the write pipeline.

- **Search Initiation:** The user inputs the origin, destination, and preferences (e.g., "Safe Route") via the *UI Controller*. The *Network Manager* wraps these parameters into a `SearchParamsDTO` and sends a `GET /v1/trips/search` request to the *WebServer*.
- **Dedicated Search Handling:** The *WebServer* identifies the read-only nature of the request and routes it directly to the specialized *Trip Search Server* via the `ITripQuery` interface (`GET /v1/search/trips`). This path avoids the computationally intensive ingestion components, ensuring responsiveness.
- **Data Retrieval and Ranking:** The *Trip Search Server* executes optimized spatial queries against the *Score Data* and *Trips Data* stores to retrieve candidate segments. It then ranks the results based on their calculated QoE score and returns a list of `BikePathDTO` objects.
- **Visualization:** Upon receiving the results, the user selects a specific route. The *UI Controller* invokes the *Map Manager* to render the path geometry (contained in the DTO) on the interactive map, using color-coding to indicate the safety status of different segments.

2.4.5. Manual Obstacle Reporting

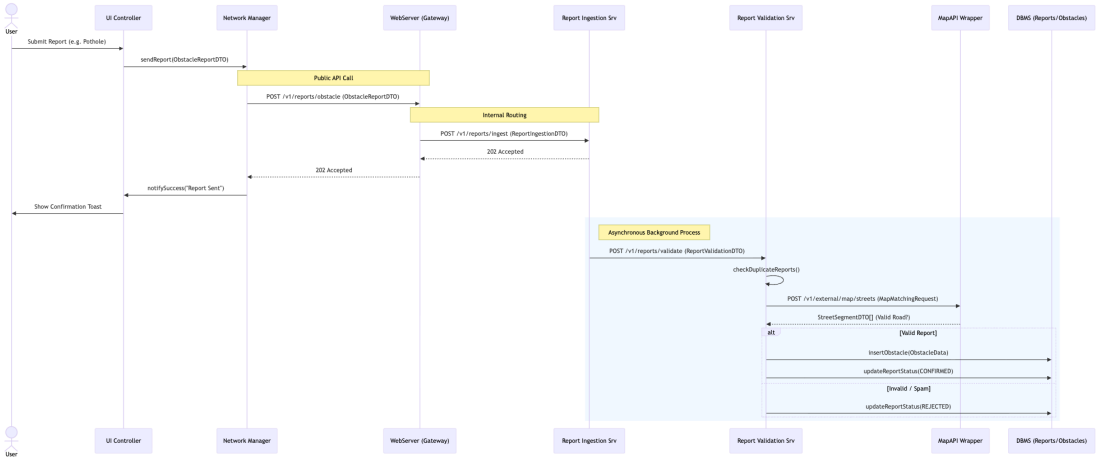


Figure 2.10: Runtime View - Manual Obstacle Reporting

Figure 2.10 details the interaction flow for the Manual Reporting scenario, where users contribute to the system by flagging road hazards (e.g., potholes). This process is managed by the dedicated *Report Server* to ensure that crowd-sourced data ingestion does not impact the performance of real-time trip tracking.

- **Submission:** The user fills out the report details (type, severity, description) via the *UI Controller*. The *Network Manager* wraps this information into an `ObstacleReportDTO` and transmits it to the *WebServer* via a secure `POST /v1/reports/obstacle` request.
- **Ingestion and Acknowledgment:** The *WebServer* forwards the request to the internal *Report Ingestion Service* (mapping it to the `IReportManagement` interface). To maintain a responsive user experience, the service immediately queues the task and returns a `202 Accepted` status code. This allows the mobile app to confirm the action to the user without waiting for the validation process.
- **Asynchronous Validation:** In the background, the *Report Ingestion Service*

wraps the data into a `ReportValidationDTO` and triggers the *Report Validation Service*. This component verifies the spatial consistency of the report by querying the external *Streets Service* (Map Matching) to ensure the coordinates correspond to a valid road segment.

- **Persistence:** Upon successful validation, the service persists the confirmed obstacle in the *Reports Data* store using the `IDataPersistence` interface, making the hazard visible to other cyclists in future searches.

2.4.6. User Authentication

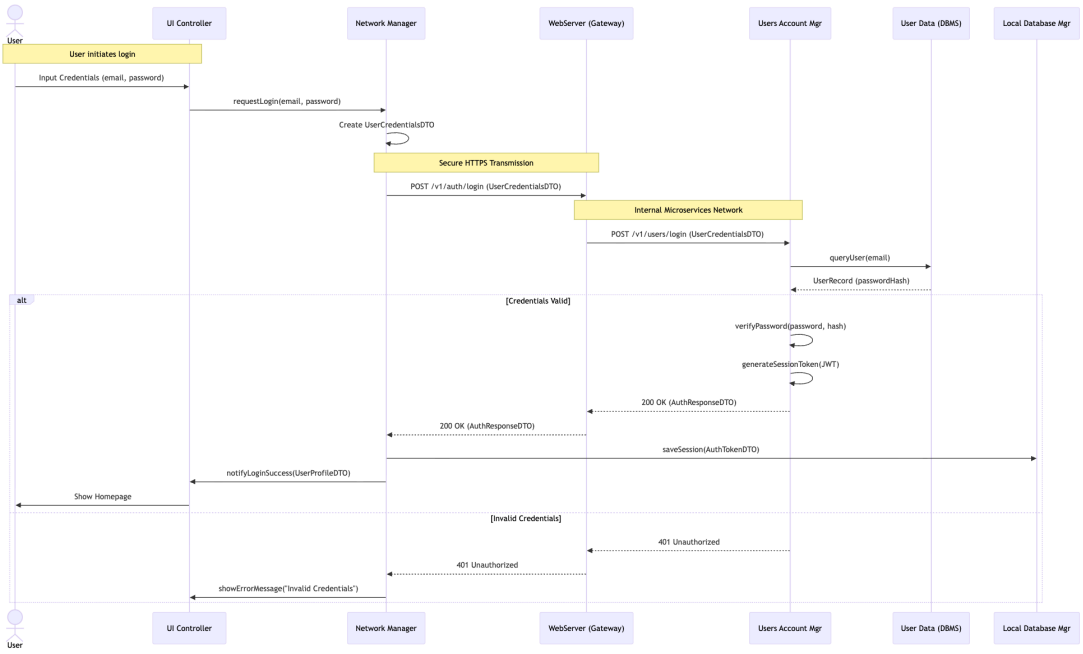


Figure 2.11: Runtime View - User Authentication

Figure 2.12 illustrates the authentication workflow for registered users. This process is managed by the specialized *Users Account Manager Service* to ensure the secure handling of sensitive credentials, isolated from the trip processing logic.

- **Credential Submission:** The user inputs their credentials via the *UI Controller*. The *Network Manager* wraps the inputs into a `UserCredentialsDTO` and transmits it to the *WebServer* (API Gateway) via a secure HTTPS channel.

- **Authentication Logic:** The *WebServer* delegates the verification request to the *Users Account Manager Service* via the `IUserManagement` interface (specifically calling the `POST /users/login` endpoint). This component retrieves the stored user hash from the *User Data* store and validates the provided password.
- **Session Management:** Upon successful verification, the service generates a secure authentication token (JWT) encapsulated in an `AuthResponseDTO`. The *Mobile App* persists this token locally via the *Local Database Manager* to maintain the session state for subsequent API calls (such as trip uploads or reporting).

2.4.7. User Registration

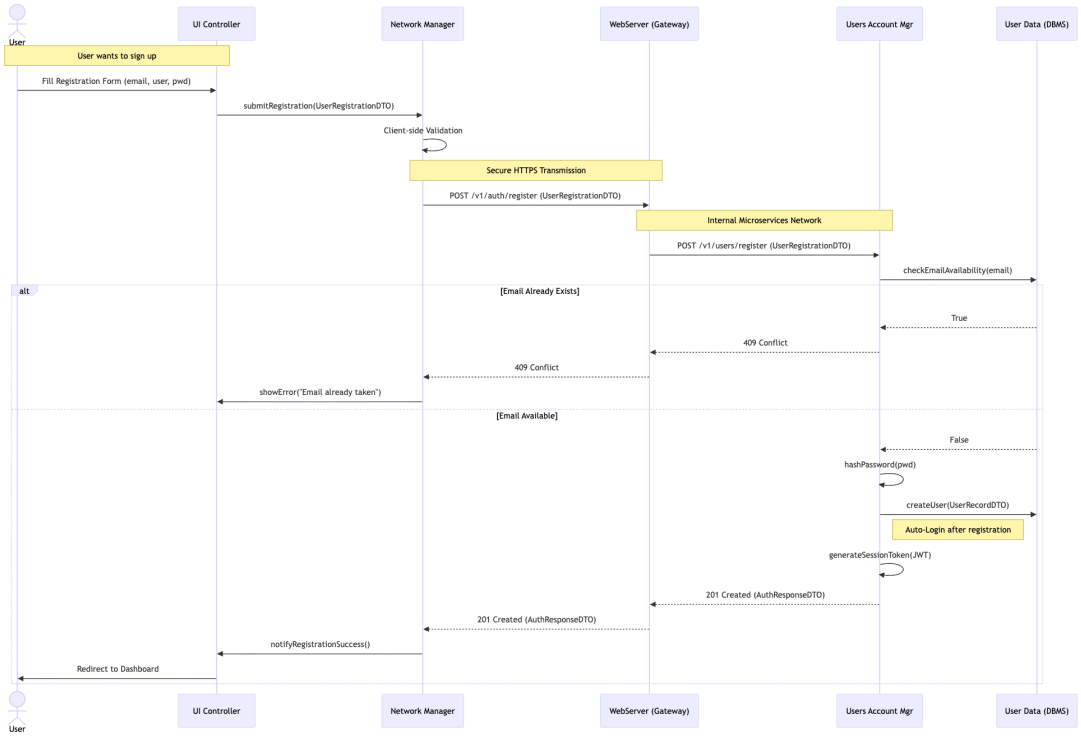


Figure 2.12: Runtime View - User Registration

This scenario details the onboarding process for a new user. The workflow ensures data consistency by preventing duplicate accounts and securing user credentials before persistence.

- **Data Submission**: The user submits their details via the `UI Controller`. The `Network Manager` validates the input format (e.g., email structure) and wraps the data into a `UserRegistrationDTO`.

- **Account Creation**: The API Gateway forwards the request to the *Users Account Manager* (`POST /v1/users/register`). The service first queries the *User Data* store to verify that the email address is unique.
- **Persistence and Auto-Login**: If the email is available, the service hashes the password and persists the new account. Upon success, it automatically generates an authentication token (JWT), returning an `AuthResponseDTO` (HTTP 201 Created) to allow immediate access without requiring a separate login step.

2.4.8. Manual Trip Creation

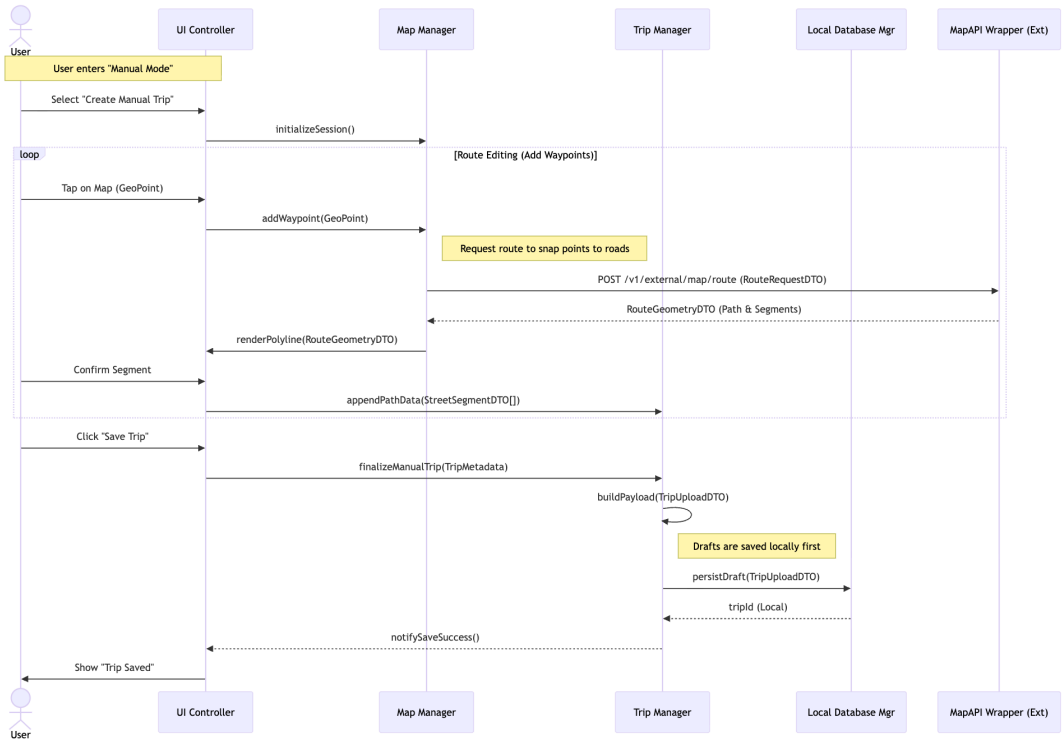


Figure 2.13: Runtime View - Manual Trip Creation

Figure 2.13 illustrates the workflow for the *Manual Trip Creation* scenario. This feature allows users to define a route and flag obstacles without physically traveling the path, relying on map interactions rather than sensor telemetry.

- **Initialization:** The user enters the creation mode via the *UI Controller*. The *Map Manager* initializes a new session to handle user inputs on the map interface.
- **Map Interaction and Snapping:** The user interacts with the map to define

waypoints (e.g., origin and destination). The *Map Manager* captures these coordinates and invokes the external map provider via the *MapAPI Wrapper* (specifically calling `POST /v1/external/map/route` with a `RouteRequestDTO`). This step "snaps" the raw points to the actual road network geometry, returning a precise `RouteGeometryDTO`.

- **Obstacle Injection:** The user can manually inject obstacle data (e.g., marking a known pothole on the drawn path). These inputs are captured by the *Trip Manager* and associated with the corresponding road segment in the draft.
- **Persistence:** Once the route is confirmed, the *Trip Manager* aggregates the segments into a `TripUploadDTO` (setting the `AcquisitionMode` to MANUAL) and persists it via the *Local Database Manager*. The trip is saved as a local draft, ready to be uploaded using the standard synchronization pipeline described in Section 2.4.2.

2.5. Component Interfaces

This section details the application programming interfaces (APIs) exposed by the system components. The data types referenced in the parameters (e.g., `PathStatus`, `GeoPoint`, `AcquisitionMode`) strictly adhere to the Domain Model defined in the RASD.

Where multiple attributes need to be transferred simultaneously, we define Data Transfer Objects (DTOs) to encapsulate the payload.

Backend Services Interfaces

The following interfaces define the interaction points for the server-side microservices.

Interface: IBBPClientAPI This interface is exposed to mobile and web clients via the API Gateway.

- **POST** `/v1/auth/register`
  - **Description:** Registers a new user account in the system.
  - **Request DTO:** `UserRegistrationDTO` (email, username, password).
  - **Response:** `UserProfileDTO` and authentication token upon success.
  - **Client:** Mobile App (Network Manager)
  - **Server:** Users Account Manager Service

- **POST** `/v1/auth/login`
  - **Description:** Authenticates a user and initiates a session.
  - **Request DTO:** `UserCredentialsDTO` (email, password).
  - **Response:** `AuthResponseDTO` containing a JWT token and `UserProfileDTO`.
  - **Client:** Mobile App (Network Manager)
  - **Server:** Users Account Manager Service
- **POST** `/v1/trips/upload`
  - **Description:** Uploads a complete trip (manual or automatic) for ingestion and processing.
  - **Request DTO:** `TripUploadDTO` (containing `Trip` data, `TrackPoint[]`, optional `WeatherConditions`).
  - **Response:** `202 Accepted` with a `tripId` for asynchronous processing.
  - **Client:** Mobile App (Network Manager)
  - **Server:** Trip Ingestion Service
- **PUT** `/v1/trips/{tripId}/publish`
  - **Description:** Publishes a previously saved draft trip, making it public and triggering aggregation.
  - **Request DTO:** `TripPublishDTO` (e.g., desired public name).
  - **Response:** `200 OK` with publication status.
  - **Client:** Mobile App (Network Manager)
  - **Server:** Trip Aggregation Service
- **GET** `/v1/trips/search`
  - **Description:** Searches for public bike paths based on geographic criteria and filters.
  - **Query Params:** `SearchParamsDTO` (origin, destination, minScore, etc.).
  - **Response:** `BikePathDTO[]` with summary information.
  - **Client:** Mobile App or Web Client (Network Manager/UI Controller)
  - **Server:** Trip Search Server

- **GET** `/v1/trips/{tripId}`
  - **Description:** Retrieves detailed information for a specific trip (private for owners, public for published trips).
  - **Response:** `TripDetailsDTO` (full trip data, path, obstacles, weather).
  - **Client:** Mobile App (Network Manager)
  - **Server:** Trip Search Server
- **POST** `/v1/reports/obstacle`
  - **Description:** Submits a manual obstacle report from a user.
  - **Request DTO:** `ObstacleReportDTO` (location, description, type, optional image).
  - **Response:** `202 Accepted` with a `reportId`.
  - **Client:** Mobile App (Network Manager)
  - **Server:** Report Ingestion Service
- **POST** `/v1/reports/path-status`
  - **Description:** Submits a manual rating/status report for a specific bike path.
  - **Request DTO:** `PathStatusReportDTO` (pathId, status, comment).
  - **Response:** `202 Accepted`.
  - **Client:** Mobile App (Network Manager)
  - **Server:** Report Ingestion Service
- **PUT** `/v1/profile`
  - **Description:** Updates the profile of the authenticated user.
  - **Request DTO:** `UserProfileUpdateDTO` (non-sensitive user information).
  - **Response:** Updated `UserProfileDTO`.
  - **Client:** Mobile App (Network Manager)
  - **Server:** Users Account Manager Service

Interface: ITripOperations (Used by Trip Ingestion Service)

- **POST** `/v1/ingestion/trips`

  - **Description:** Internal endpoint for receiving raw trip data. Performs initial validation and triggers the processing pipeline.
  - **Request DTO:** `TripUploadDTO`.
  - **Response:** `202 Accepted`.
  - **Client:** WebServer (API Gateway)
  - **Server:** Trip Ingestion Service

Interface: ITripCompliance (Used by Trip Validation Service)

- **POST** `/v1/validation/trips`
  - **Description:** Validates trip data for consistency, filters GPS noise, and performs map-matching.
  - **Request DTO:** `TripUploadDTO`.
  - **Response:** `ValidatedTripDTO` or a list of validation errors.
  - **Client:** Trip Ingestion Service
  - **Server:** Trip Validation Service

Interface: ITripAggregation (Used by Trip Aggregation Service)

- **POST** `/v1/aggregation/trips`
  - **Description:** Merges validated trip data with the historical road state and runs consensus algorithms.
  - **Request DTO:** `ValidatedTripDTO`.
  - **Response:** `AggregationResultDTO` (summary of applied updates).
  - **Client:** Trip Validation Service
  - **Server:** Trip Aggregation Service

Interface: IScoringCalculation (Used by Trip Scoring Service)

- **POST** `/v1/scoring/calculate`
  - **Description:** Calculates or updates the quality and safety score for affected road segments.

  - **Request DTO:** `AggregatedDataDTO` (from aggregation service).
  - **Response:** `ScoreUpdateDTO` (new scores for paths).
  - **Client:** Trip Aggregation Service
  - **Server:** Trip Scoring Service

Interface: IReportManagement (Used by Report Server)

- **POST** `/v1/reports/ingest`
  - **Description:** Internal endpoint for ingesting user-generated reports.
  - **Request DTO:** `ReportIngestionDTO` (wraps `ObstacleReportDTO` or `PathStatusReportDTO`
  - **Response:** `202 Accepted`.
  - **Client:** WebServer (API Gateway)
  - **Server:** Report Ingestion Service
- **POST** `/v1/reports/validate`
  - **Description:** Validates an ingested report (spatial consistency, duplicates) and updates its state.
  - **Request DTO:** `ReportValidationDTO`.
  - **Response:** `ValidationResultDTO`.
  - **Client:** Report Ingestion Service
  - **Server:** Report Validation Service

Interface: ITripQuery (Used by Trip Search Server)

- **GET** `/v1/search/trips`
  - **Description:** Executes complex read-only queries for bike paths. Optimized for low latency.
  - **Query Params:** `SearchParamsDTO`.
  - **Response:** `BikePathDTO[]`.
  - **Client:** WebServer (API Gateway)
  - **Server:** Trip Search Server

Interface: IUserManagement (Used by Users Account Manager Service)

- **POST** `/v1/users/register`
  - **Description:** Internal endpoint for creating a new user account.
  - **Request DTO:** `UserRegistrationDTO`.
  - **Response:** `UserRecordDTO` (internal user representation).
  - **Client:** WebServer (API Gateway)
  - **Server:** Users Account Manager Service
- **POST** `/v1/users/login`
  - **Description:** Internal endpoint for authenticating a user.
  - **Request DTO:** `UserCredentialsDTO`.
  - **Response:** `AuthTokenDTO` (JWT) and `UserRecordDTO`.
  - **Client:** WebServer (API Gateway)
  - **Server:** Users Account Manager Service

External Wrappers Interfaces

These interfaces act as adapters for third-party APIs, decoupling the core system from external providers.

Interface: IMeteoProvider (Used by WeatherAPI Wrapper)

- **POST** `/v1/external/weather/conditions`
  - **Description:** Retrieves historical or real-time weather conditions for a given location and time.
  - **Request DTO:** `WeatherQueryDTO` (GeoPoint, timestamp).
  - **Response:** `WeatherConditionsDTO`.
  - **Client:** Trip Ingestion Service / Trip Validation Service
  - **Server:** WeatherAPI Wrapper (calls external provider)

Interface: IMapProvider (Used by MapAPI Wrapper)

- **POST** `/v1/external/map/streets`

  - **Description:** Performs map-matching, returning the closest street segment ID for a given coordinate.
  - **Request DTO:** `MapMatchingRequestDTO` (GeoPoint or TrackPoint[]).
  - **Response:** `StreetSegmentDTO[]`.
  - **Client:** Trip Validation Service / Report Validation Service / Map Manager
  - **Server:** MapAPI Wrapper (calls OpenStreetMap/other)
- **POST** `/v1/external/map/route`
  - **Description:** Gets the route geometry (polyline) between two points.
  - **Request DTO:** `RouteRequestDTO` (origin GeoPoint, destination GeoPoint).
  - **Response:** `RouteGeometryDTO`.
  - **Client:** Map Manager (for manual trip creation)
  - **Server:** MapAPI Wrapper

Data Persistence Interface

Interface: IDataPersistence

- **Description:** This interface defines the contract for data access operations. It is typically implemented as a set of repository patterns or database driver calls (e.g., SQL, NoSQL client). Each service (Trip Ingestion, Aggregation, Scoring, Report, User Manager) uses this interface to persist and retrieve its specific data entities (`Trip`, `Obstacle`, `Score`, `Report`, `User`) from the respective databases in the DBMS cluster.

Utility & Monitoring Interfaces

- **GET** `/health`
  - **Description:** Standard health check endpoint for load balancers and orchestration (Kubernetes).
  - **Response:** Simple JSON with status `{"status": "UP"}`.
  - **Client:** Kubernetes, Load Balancer
  - **Server:** All Services

- **GET** `/metrics`
  - **Description:** Exposes application metrics in a format consumable by monitoring systems like Prometheus.
  - **Response:** Prometheus metrics data.
  - **Client:** Prometheus Server
  - **Server:** All Services (optional)

2.5.1. Data Transfer Objects (DTOs)

This section defines the structure of all Data Transfer Objects (DTOs) used in the API interfaces. Each DTO is designed to encapsulate the data required for a specific operation, ensuring type safety and a clear contract between clients and services.

Authentication and User Management DTOs

- `UserRegistrationDTO`
  - **Description:** Carries the necessary information to register a new user.
  - **Fields:**
    - `email: String` – The user's email address.
    - `username: String` – The chosen username.
    - `password: String` – The user's plain-text password (to be hashed server-side).
- `UserCredentialsDTO`
  - **Description:** Contains credentials for user authentication.
  - **Fields:**
    - `email: String` – The user's email address.
    - `password: String` – The user's password.
- `UserProfileDTO`
  - **Description:** Represents a user's public profile information.
  - **Fields:**

        * `userId: ID` – Unique user identifier.
        * `username: String` – The user's display name.
        * `email: String` – The user's email address.
        * `registrationDate: DateTime` – Date and time of account creation.
        * `preferences: UserPreferences` – User-specific application settings (optional).

- `UserProfileUpdateDTO`
  - **Description:** Contains fields that can be updated in a user's profile.
  - **Fields:**
    * `username: String` (optional) – New display name.
    * `preferences: UserPreferences` (optional) – Updated application settings.

- `AuthResponseDTO`
  - **Description:** Returned upon successful authentication.
  - **Fields:**
    * `token: String` – A JSON Web Token (JWT) for authenticated session.
    * `user: UserProfileDTO` – The authenticated user's profile.

- `UserRecordDTO`
  - **Description:** Internal representation of a user record, including sensitive information that is not exposed to clients.
  - **Fields:**
    * `userId: ID`
    * `username: String`
    * `email: String`
    * `passwordHash: String` – The hashed password.
    * `registrationDate: DateTime`
    * `lastLogin: DateTime` (optional)

        * `accountStatus: AccountStatus` (e.g., ACTIVE, SUSPENDED).

- `AuthTokenDTO`
  - **Description:** Internal DTO containing authentication token data.
  - **Fields:**
    * `token: String` – The JWT string.
    * `expiresAt: DateTime` – Token expiration timestamp.

Trip and Path Related DTOs

- `TripUploadDTO`
  - **Description:** Contains all data for a complete trip, either recorded automatically or created manually.
  - **Fields:**
    * `tripId: ID` (optional) – Client-generated unique identifier for the trip draft.
    * `acquisitionMode: AcquisitionMode` – MANUAL or AUTOMATIC.
    * `startTime: DateTime`
    * `endTime: DateTime`
    * `distance: Double` – Total distance in kilometers.
    * `avgSpeed: Double` – Average speed in km/h.
    * `trackPoints: TrackPoint[]` – Ordered sequence of geographic samples.
    * `weatherConditions: WeatherConditions` (optional) – Weather data for the trip.
    * `obstacleReports: ObstacleReportDTO[]` (optional) – Obstacles detected or reported during the trip.
- `ValidatedTripDTO`
  - **Description:** Represents a trip that has passed initial validation and map-matching.
  - **Fields:** Inherits all fields from `TripUploadDTO` and adds:

    * `validationStatus: ValidationStatus` – e.g., VALID, INVALID.
    * `matchedStreetSegments: StreetSegmentDTO[]` – The result of map-matching, linking each track point or segment to a street.
    * `validationErrors: String[]` (optional) – If status is INVALID, contains error messages.

- `TripPublishDTO`
  - **Description:** Contains information required to publish a draft trip.
  - **Fields:**
    * `name: String` – A user-assigned public name for the path.
    * `description: String` (optional) – A longer description.
    * `isPublic: Boolean` – Flag to control visibility (default: true).

- `TripDetailsDTO`
  - **Description:** Contains comprehensive information about a specific trip, intended for detailed viewing.
  - **Fields:**
    * All fields from `TripUploadDTO`
    * `bikePath: BikePathDTO` (optional) – If the trip has been published, the associated public path.
    * `owner: UserProfileDTO` – Profile of the trip's owner.
    * `createdAt: DateTime` – When the trip was first saved.
    * `lastModified: DateTime` – When the trip was last edited.

- `BikePathDTO`
  - **Description:** Represents a public bike path, aggregated from user contributions.
  - **Fields:**
    * `pathId: ID` – Unique identifier for the public path.
    * `name: String` – Public name of the path.
    * `origin: GeoPoint`

    * `destination: GeoPoint`
    * `score: Double` – Current quality/safety score.
    * `status: PathStatus` – Aggregated status (e.g., OPTIMAL, MEDIUM).
    * `streets: StreetSegmentDTO[]` – The sequence of streets that compose the path.
    * `obstacles: ObstacleDTO[]` – Confirmed obstacles along the path.
    * `lastUpdated: DateTime` – Timestamp of the last update to the path's data.

- `SearchParamsDTO`
  - **Description:** Encapsulates criteria for searching bike paths.
  - **Fields:**
    * `origin: GeoPoint` – Starting location.
    * `destination: GeoPoint` – Target location.
    * `maxDistance: Double` (optional) – Maximum acceptable route distance.
    * `minScore: Double` (optional) – Minimum acceptable path score.
    * `statusFilter: PathStatus[]` (optional) – List of acceptable path statuses.

- `AggregatedDataDTO`
  - **Description:** Contains data that has been merged from multiple trips for a specific road segment, ready for scoring.
  - **Fields:**
    * `streetSegmentId: ID` – Identifier of the street segment.
    * `averageRoughness: Double` – Merged surface roughness metric.
    * `averageSpeed: Double` – Merged average speed.
    * `confirmedObstacles: ObstacleDTO[]` – List of confirmed obstacles on this segment.
    * `userReports: PathStatusReportDTO[]` – Collection of user-submitted status reports for the segment.

    * `dataFreshness: DateTime` – Timestamp of the most recent data point.

- `ScoreUpdateDTO`
  - **Description:** Contains the result of a scoring calculation for one or more path segments.
  - **Fields:**
    * `updates: Map<ID, Double>` – A map where the key is a path or segment ID and the value is the new calculated score.
    * `calculationTimestamp: DateTime` – When the calculation was performed.
- `AggregationResultDTO`
  - **Description:** Summary of changes applied during the aggregation of a new trip.
  - **Fields:**
    * `tripId: ID` – The ID of the trip that was aggregated.
    * `updatedSegments: ID[]` – List of street segment IDs that were updated.
    * `newObstaclesConfirmed: Integer` – Number of new obstacles confirmed by the voting engine.
    * `statusChanges: Map<ID, PathStatus>` – Any changes to the aggregated status of affected paths.

Reporting DTOs

- `ObstacleReportDTO`
  - **Description:** Represents a single obstacle report, either manually submitted or automatically detected.
  - **Fields:**
    * `location: GeoPoint` – Geographic coordinates of the obstacle.
    * `type: ObstacleType` – e.g., POTHOLEDEBRIS, CONSTRUCTION.
    * `description: String` (optional) – User-provided description.
    * `severity: SeverityLevel` (optional) – User-perceived severity.
    * `imageUrl: String` (optional) – Reference to an uploaded image.

    * `acquisitionMode: AcquisitionMode` – MANUAL or AUTOMATIC.
    * `tripId: ID` (optional) – If the report was generated during a trip.

- `PathStatusReportDTO`
  - **Description:** A user's qualitative report on the condition of a path or street segment.
  - **Fields:**
    * `pathId: ID` (optional) – The ID of the bike path being rated. (Mutually exclusive with `streetSegmentId`)
    * `streetSegmentId: ID` (optional) – The ID of a specific street segment being rated.
    * `status: PathStatus` – The reported condition (e.g., OPTIMAL, REQUIRES_MAINTENANCE).
    * `comment: String` (optional) – Additional user feedback.
    * `acquisitionMode: AcquisitionMode` – MANUAL or AUTOMATIC.
- `ReportIngestionDTO`
  - **Description:** Wrapper for any type of user report entering the reporting pipeline.
  - **Fields:**
    * `reportType: ReportType` – OBSTACLE or PATH_STATUS.
    * `payload: Object` – The actual report DTO (`ObstacleReportDTO` or `PathStatusReportDTO`).
    * `userId: ID` – Identifier of the reporting user.
    * `submittedAt: DateTime` – Client-side submission timestamp.
- `ReportValidationDTO`
  - **Description:** Contains a report along with contextual data needed for validation.
  - **Fields:**
    * `report: ReportIngestionDTO` – The original ingested report.

    * `validationContext: ValidationContext` – Additional data (e.g., user trust score, location accuracy).

- `ValidationResultDTO`
  - **Description:** Result of the report validation process.
  - **Fields:**
    * `reportId: ID` – The ID of the validated report.
    * `isValid: Boolean`
    * `status: ObstacleStatus` (if applicable) – The new status of the obstacle report (e.g., CONFIRMED, REJECTED, PENDING).
    * `message: String` (optional) – Explanation of the validation outcome.
    * `validatedObstacleId: ID` (optional) – If an obstacle was confirmed/created, its new ID.

External Service DTOs

- `WeatherQueryDTO`
  - **Description:** Query for historical or real-time weather data.
  - **Fields:**
    * `location: GeoPoint`
    * `timestamp: DateTime` – The point in time for which weather is requested.
    * `timeRange: Duration` (optional) – A time window around the timestamp (for historical averages).
- `WeatherConditionsDTO`
  - **Description:** Meteorological conditions at a specific location and time.
  - **Fields:**
    * `temperature: Double` – in degrees Celsius.
    * `windSpeed: Double` – in km/h.
    * `weatherStatus: WeatherStatus` – e.g., SUNNY, RAINY, CLOUDY.

    * `timestamp: DateTime` – The time these conditions were recorded.
    * `location: GeoPoint` – The location these conditions apply to.

- `MapMatchingRequestDTO`
  - **Description:** Request to snap a coordinate or a sequence of coordinates to the road network.
  - **Fields:**
    * `points: GeoPoint[]` – A single point or an ordered list of track points.
    * `requireCompleteMatch: Boolean` (optional) – If true, the request fails if any point cannot be matched.

- `StreetSegmentDTO`
  - **Description:** Represents a segment of a street in the road network.
  - **Fields:**
    * `segmentId: ID` – Unique identifier from the map provider (e.g., OpenStreetMap way ID).
    * `name: String` – Street name.
    * `geometry: GeoPoint[]` – Polyline representing the segment's shape.
    * `length: Double` – Segment length in meters.

- `RouteRequestDTO`
  - **Description:** Request for a route between two points.
  - **Fields:**
    * `origin: GeoPoint`
    * `destination: GeoPoint`
    * `preferences: RoutePreferences` (optional) – e.g., avoid highways, prefer bike lanes.

- `RouteGeometryDTO`
  - **Description:** The geometric result of a routing request.
  - **Fields:**
    * `path: GeoPoint[]` – The sequence of coordinates forming the route.

* `distance: Double` – Total route distance in meters.
* `estimatedDuration: Duration` – Estimated travel time.
* `segments: StreetSegmentDTO[]` – The list of street segments that compose the route.

Primitive and Enumeration Types

The following common types are used throughout the DTOs:

- `ID`: A string or integer representing a unique identifier.
- `GeoPoint`: An object with `latitude: Double` and `longitude: Double` fields.
- `DateTime`: ISO 8601 timestamp string.
- `Duration`: A time span represented in seconds.
- `AcquisitionMode`: Enumeration with values `MANUAL` and `AUTOMATIC`.
- `PathStatus`: Enumeration with values `OPTIMAL, MEDIUM, SUFFICIENT, REQUIRES_MAINTENANC NOT_AVAILABLE`.
- `ObstacleStatus`: Enumeration with values `PENDING, CONFIRMED, REJECTED`.
- `WeatherStatus`: Enumeration with values `SUNNY, RAINY, CLOUDY, SNOWY`.
- `TrackPoint`: An object containing `timestamp: DateTime`, `coordinates: GeoPoint`, and `elevation: Double` (optional).

2.6. Selected Architectural Styles and Patterns

The architectural design of Best Bike Paths (BBP) combines established architectural styles with modern software design patterns to meet the system's functional and non-functional requirements.

2.6.1. Microservices Architecture

BBP adopts a **Microservices Architecture**, decomposing the system into independently deployable services, each implementing a specific business capability. This approach aligns with the system's requirements for high availability, scalability, and the ability to evolve different components at different rates. The core services (Trip Ingestion, Validation, Aggregation, Scoring, Report Management, and User Management) operate

as separate processes, communicating via well-defined REST APIs.

The benefits of this choice include:

- **Independent Scalability:** Resource-intensive services (e.g., Trip Ingestion) can be scaled independently from compute-intensive ones (e.g., Trip Aggregation).
- **Fault Isolation:** A failure in one service (e.g., Report Validation) does not cascade to others (e.g., Trip Recording).
- **Technology Heterogeneity:** Different services can use technology stacks best suited to their specific requirements (e.g., time-series database for telemetry, relational database for user data).

2.6.2. Command Query Responsibility Segregation (CQRS)

To ensure optimal performance for both data ingestion and query operations, BBP implements a **CQRS (Command Query Responsibility Segregation)**-inspired pattern. The write operations (Commands), such as uploading and processing trips, are handled by the *BBP Backend Cluster* (Ingestion → Validation → Aggregation → Scoring). The read operations (Queries), such as searching for and retrieving path information, are handled by the dedicated, read-optimized *Trip Search Server*. This separation prevents resource contention between high-throughput ingestion pipelines and low-latency search queries, directly addressing the performance requirements outlined in Section 3.3 of the RASD.

2.6.3. API Gateway Pattern

The **API Gateway Pattern** is realized through the *WebServer* component. It acts as the single entry point for all client requests, providing essential cross-cutting concerns such as request routing, authentication, SSL termination, and load balancing. This pattern simplifies the client interaction by hiding the complexity of the underlying microservices topology and provides a unified interface for security and monitoring.

2.6.4. Polyglot Persistence

BBP employs a **Polyglot Persistence** strategy within its DBMS cluster. Different data stores are selected based on the nature and access patterns of the data:

- A relational database with geospatial extensions (e.g., PostgreSQL with PostGIS) for structured business data (Users, Reports) and complex geospatial queries.

- A time-series database (e.g., AWS Timestream or InfluxDB) for the high-volume, append-only streams of raw sensor telemetry.

This approach optimizes performance and storage efficiency by using the right tool for each specific data type.

2.6.5. Containerization and Orchestration

All backend services and the mobile application's server-side components are designed as **containerized** applications. **Kubernetes** is selected as the container orchestration platform to manage deployment, scaling, and service discovery. This facilitates a consistent deployment model, rapid scaling during peak usage (e.g., commuting hours), and high availability through automated health checks and self-healing.

2.7. Security and Data Privacy

Security and data privacy are foundational pillars of the Best Bike Paths (BBP) architecture. The system adopts a *Defense in Depth* approach, implementing multiple layers of security controls ranging from network transport to data persistence, ensuring compliance with GDPR regulations and protecting the integrity of the community-driven data.

2.7.1. Network and Communication Security

To guarantee the confidentiality and integrity of data in transit, specifically regarding telemetry streams and user credentials, the following measures are enforced:

- **Transport Layer Security (TLS):** All communications between client applications (Mobile and Web) and the backend infrastructure are encrypted using **TLS 1.3**. The *WebServer* (acting as the API Gateway) handles SSL Termination, decrypting traffic before routing it to the internal microservices network.
- **Certificate Pinning (Mobile App):** To mitigate Man-in-the-Middle (MitM) attacks on public networks, the mobile application implements certificate pinning. It trusts only the specific public key or certificate of the BBP backend, rejecting any other certificate authority, even if trusted by the device OS.
- **API Gateway Protection:** The Gateway is configured with strict **Rate Limiting** policies (e.g., per-IP request caps) to protect the system against Distributed Denial of Service (DDoS) attacks and brute-force login attempts.

2.7.2. Authentication and Identity Management

Authentication is managed centrally by the *Users Account Manager*, ensuring stateless and scalable identity verification.

- **JSON Web Tokens (JWT):** Upon successful login, the system issues a JWT signed using an asymmetric algorithm (e.g., **RS256**). This ensures that microservices can validate the token's authenticity using the public key without needing to query the Account Manager for every request.
- **Password Hashing:** User passwords are never stored in plain text. The system employs memory-hard hashing algorithms (such as **Argon2id** or **BCrypt**) with unique salts for each user, providing robust protection against rainbow table attacks and GPU-accelerated cracking.

2.7.3. Data Privacy and GDPR Compliance

Given the sensitive nature of location data, the architecture includes specific mechanisms to preserve user privacy and adhere to GDPR "Privacy by Design" principles.

- **Separation of Concerns and PII Isolation:** Personally Identifiable Information (PII) such as email and names is stored exclusively in the *User Data* store. The *Trips Data* and *Obstacles Data* stores reference users only via opaque unique identifiers (UUIDs). This allows for effective pseudo-anonymization: if a user exercises their "Right to be Forgotten," their PII can be deleted while retaining the anonymous trip data for community statistics.
- **Geo-Obfuscation (Privacy Zones):** To prevent the inference of sensitive locations (e.g., home or workplace addresses), the system applies a geo-obfuscation logic. When a trip is published, the start and end coordinates are truncated or fuzzed within a specific radius (e.g., 200 meters) before being visible to other users.
- **Encryption at Rest:** All persistent data stores (DBMS Cluster) are configured with encryption at the volume level (e.g., AES-256), ensuring data remains unreadable even in the event of physical hardware theft or improper decommissioning.

2.7.4. Infrastructure Security

Leveraging the containerized nature of the deployment (Kubernetes), infrastructure security is managed as code:

- **Secrets Management:** Sensitive configuration data (database credentials, API keys for Weather and Map providers, JWT private keys) are never hardcoded. They are injected at runtime using **Kubernetes Secrets** or dedicated secret management vaults.
- **Principle of Least Privilege:** Microservices containers operate with non-root privileges and read-only filesystems where possible, minimizing the potential impact of a container compromise.
- **Network Policies:** Internal traffic is governed by strict network policies. Services can only communicate with the specific components required for their function (e.g., the *Trip Search Server* cannot communicate directly with the *Report Server*), limiting lateral movement in case of a breach.

2.8. Other Design Decisions

2.8.1. Asynchronous Processing and Eventual Consistency

To maximize user experience quality (QoE), the system is designed for **asynchronous processing**. Critical user interactions, such as trip uploads and report submissions, receive immediate acknowledgment (HTTP 202 Accepted) while the heavy computational work (validation, aggregation, scoring) proceeds in the background. This design prioritizes responsiveness and allows the system to gracefully handle traffic spikes, accepting that the global state of path scores and conditions is updated with a slight, acceptable delay (eventual consistency).

2.8.2. Dedicated External Service Adapters

All interactions with external providers (OpenStreetMap, Weather Services) are abstracted behind dedicated **Adapter/Wrapper** components (e.g., MapAPI Wrapper, WeatherAPI Wrapper). This design decision encapsulates third-party API specifics, provides a consistent internal interface, and centralizes logic for error handling, retries, and potential fallback mechanisms, making the system more resilient to external service failures.

2.8.3. Client-Side Data Persistence and Resilience

The mobile application includes a **Local Database Manager** component that persists trip data immediately upon collection. This design ensures data integrity in case of application crashes or network loss and enables a seamless offline-to-online synchronization

experience. It aligns with the constraint (C05) that the platform should remain functional with intermittent connectivity.

2.8.4. Modular Mobile Application Architecture

The mobile application is decomposed into distinct modules (UI Controller, Sensor Manager, Trip Manager, Network Manager, Map Manager). This modular design enhances maintainability, allows for easier testing of individual components, and provides a clear separation between the presentation layer, business logic, and hardware abstraction, facilitating future updates or platform-specific optimizations.

3 | User Interface Design

This section presents the User Interface (UI) and User Experience (UX) design of the Best Bike Paths (BBP) system. Since BBP is primarily a mobile-based application, the interface is designed following a mobile-first approach, taking into account the constraints and interaction patterns of smartphone usage. The goal of the UI/UX design is to enable users to access the system's functionalities in a simple, safe, and efficient manner, even in contexts where attention and interaction time may be limited.

The design emphasizes clarity, consistency, and ease of navigation. Screens are structured to minimize cognitive load and to present only the most relevant information at each step of the user interaction. Actions commonly performed while cycling, as well as shortly before and after a trip, are designed to require a minimal number of gestures and to provide immediate and clear feedback.

The interface is illustrated through a set of mobile application mockups created using Figma as a design tool. These mockups follow the main user flows of the system and represent the expected layout and functional organization of the application. They are used to explain how users interact with BBP during typical usage scenarios. Separate interaction flows are described for different user types, reflecting the distinction between registered and unregistered users defined in the RASD.

For the sake of clarity, not all application screens are shown. Simple or self-explanatory views, as well as screens that closely resemble others in structure or behavior, are omitted from the document. When this occurs, the missing steps in the user flow are briefly described in text to preserve continuity and completeness.

The included screenshots correspond to a mockup version of the mobile application developed in Figma and are intended to represent the structural and functional aspects of the interface. They do not define the final graphical style of the system and may omit or simplify specific visual elements such as colors, icons, and detailed graphics. These aspects are intentionally abstracted in order to focus on interaction flow, layout, and usability, which are the primary concerns at this stage of the design.

3.1. User Flow Diagram

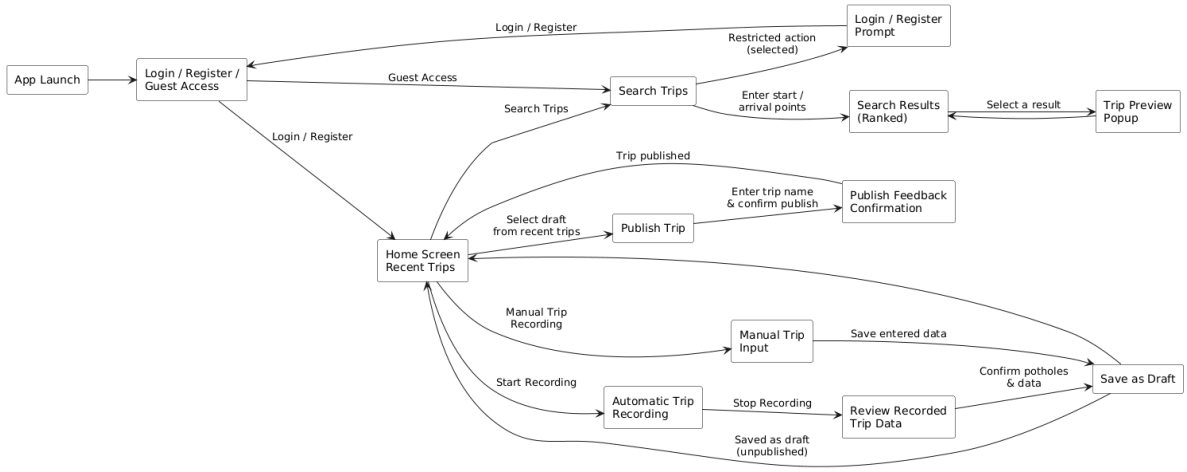


Figure 3.1: Unified User Flow Diagram for the BBP Mobile Application

Figure 3.1 illustrates the unified user flow of the Best Bike Paths (BBP) mobile application, capturing the main interaction paths for both registered cyclists and unregistered users (guests). The diagram highlights how users navigate through the application starting from the initial access screen, and how available functionalities differ based on authentication status.

Upon launching the application, users are presented with an initial screen that allows them to either log in, register a new account, or continue as a guest. Users who authenticate or register successfully are redirected to the Home Screen, which serves as the central hub of the application. From this screen, registered cyclists can access the core functionalities of BBP, including automatic trip recording, manual trip data entry, and trip search.

When starting an automatic recording session, the user can stop the recording at any time and is redirected to a review screen where collected data is displayed. At this stage, the cyclist can validate detected events, such as potholes, and save the trip as a draft. Draft trips appear in the Home Screen under recent trips and can be further edited or published. Publishing a trip requires explicit confirmation, including the insertion of a trip name, and provides clear feedback upon completion of the action.

Manual trip recording follows a similar workflow, allowing users to insert trip data directly without automatic tracking. Manually entered trips are also saved as drafts and can be published at a later time through the same confirmation process.

Both registered and unregistered users can access the trip search functionality. Users

may specify a starting point and a destination to retrieve a ranked list of available trips. Selecting a result displays a brief preview with summarized information about the trip. This interaction remains read-only for unregistered users.

Guest users are intentionally limited to browsing and searching for trips. Any attempt to access restricted functionalities, such as trip recording, draft management, or publishing, triggers a prompt inviting the user to log in or register. This mechanism ensures a smooth transition from guest usage to full account creation while preserving a low-friction initial experience.

3.1.1. Login

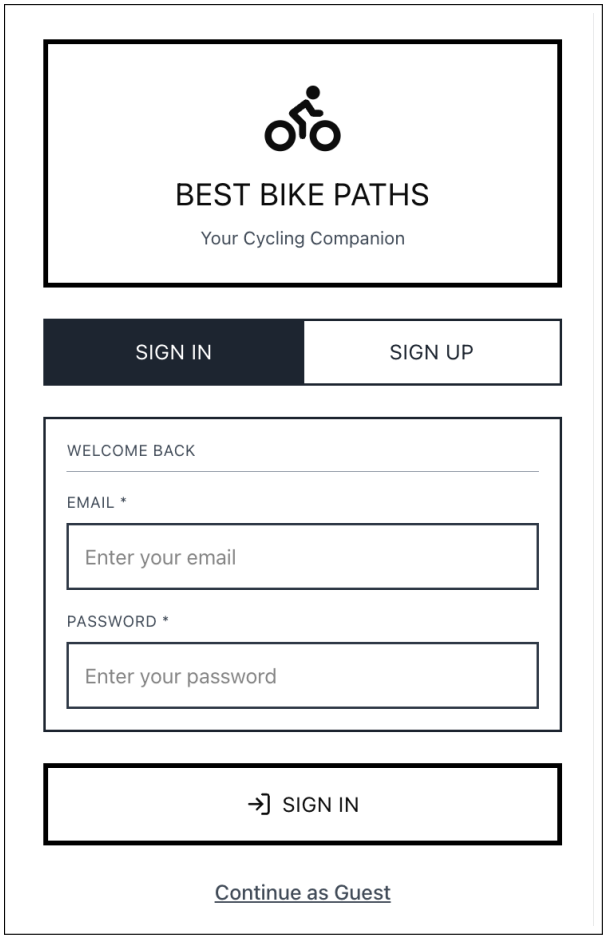


Figure 3.2: Login Page - ST

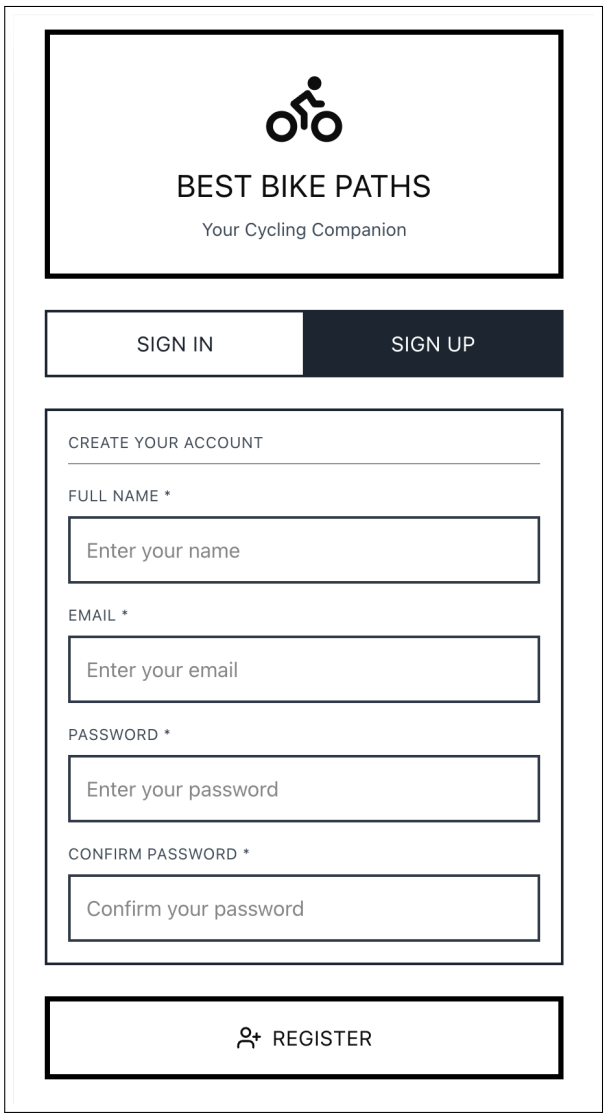


Figure 3.3: Signup Page - ST

The BBP login page serves as the initial entry point for the platform. Users are presented with three primary options: they can authenticate using previously created credentials, register a new account by providing their name, email, and password, or choose to "Continue as Guest." This final option allows unregistered users to immediately access the public map and navigation features without authentication, satisfying the requirement for open access to the bike path app.

3.1.2. User Profile

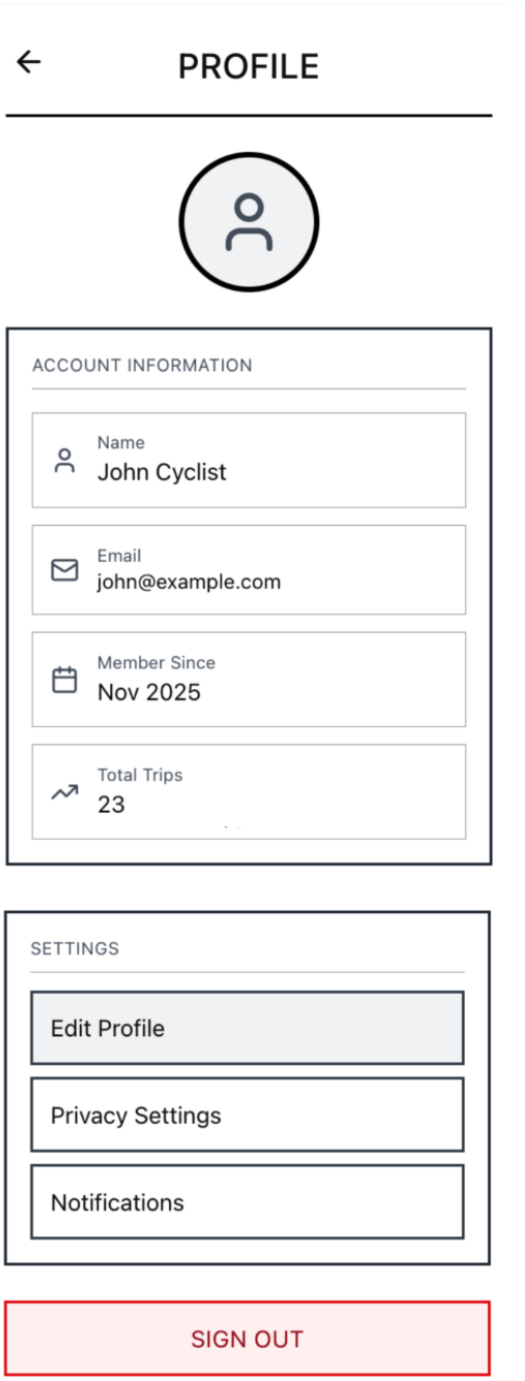


Figure 3.4: Mockup of the BBP User Profile UI

The Profile interface is accessible via the user icon located in the global header. It serves as the central hub for managing the registered user's account details and preferences.

The upper panel, "Account Information," identifies the user and displays key membership details. Notably, it includes a "Total Trips" counter, which provides a high-level summary of the user's engagement, consistent with the system's requirement to store and track cycling activities over time.

The lower section focuses on account configuration. The "Settings" menu allows users to modify their personal details via "Edit Profile" and manage application preferences such as "Notifications." Crucially, the "Privacy Settings" option is available to control data visibility, which is particularly relevant given that users can choose to publish or keep their trip data private. Finally, a prominent "Sign Out" button at the bottom ensures users can securely terminate their session.

3.1.3. Homepage

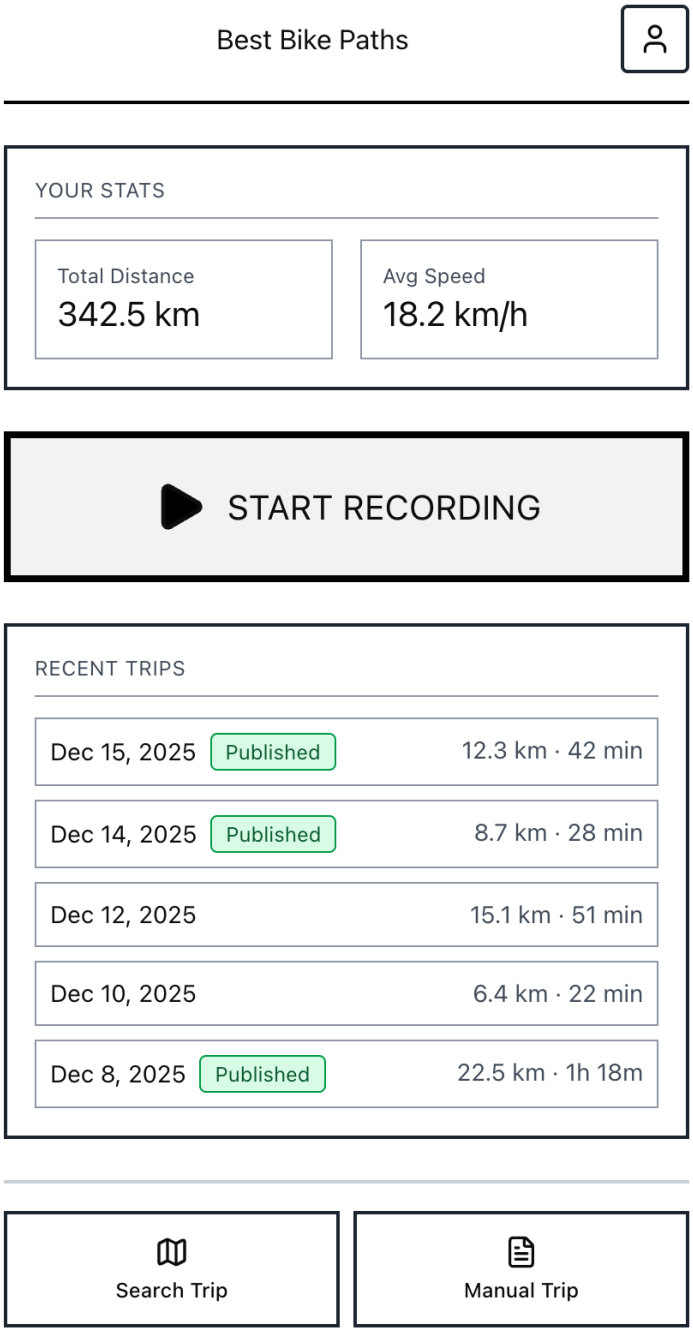


Figure 3.5: Mockup of the BBP Homepage UI

The homepage is the main entry point for the BBP application. Here the user can view a summary of their cycling performance, specifically their total distance covered and average

speed. As with all other pages, the header is present and allows the user to access their profile to manage personal settings or log out.

The central section features a prominent "Start Recording" button, enabling the user to instantly initiate the *Automated Mode*, where the system acquires data from the mobile device sensors while biking. Below the main action area, a "Recent Trips" section lists the user's history, displaying key details such as date, distance, duration, and the publication status (e.g., "Published") for each trip. Finally, the footer navigation provides quick access to two other core functionalities: "Search Trip," allowing users to visualize bike paths on a map between an origin and destination, and "Manual Trip," which enables the manual insertion of trip data and path status.

3.1.4. Auto Recording Trip Interface

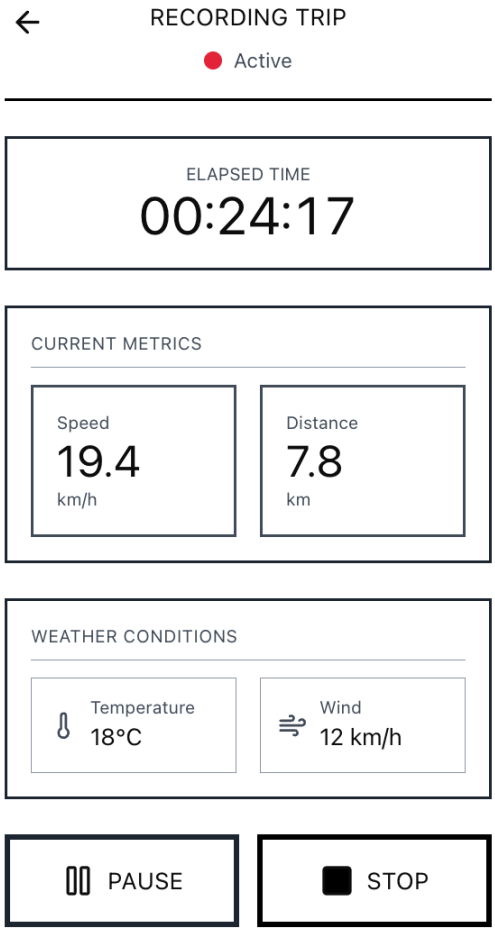


Figure 3.6: BBP Auto Recording Trip UI

This interface is activated immediately after the user selects "Start Recording" from the homepage. It represents the active state of the *Automated Mode*, providing the user with real-time monitoring of their current activity. A status indicator at the top confirms that the system is actively tracking the session.

The central dashboard is divided into three key informational areas. The top section displays the "Elapsed Time" since the start of the trip. Below this, the "Current Metrics" panel provides live feedback on the user's performance, specifically showing the current speed and the total distance covered so far, derived from the device's GPS and sensor data.

Additionally, the application enriches the trip context through the "Weather Conditions" panel. As specified in the system requirements, this section displays real-time meteorological data, such as temperature and wind speed, retrieved from external services. Finally, the footer offers "Pause" and "Stop" controls, allowing the user to manage the recording session. Pressing "Stop" will conclude the data acquisition and proceed to the verification stage where the user can confirm the reconstructed path.

3.1.5. Trip Completion and Verification

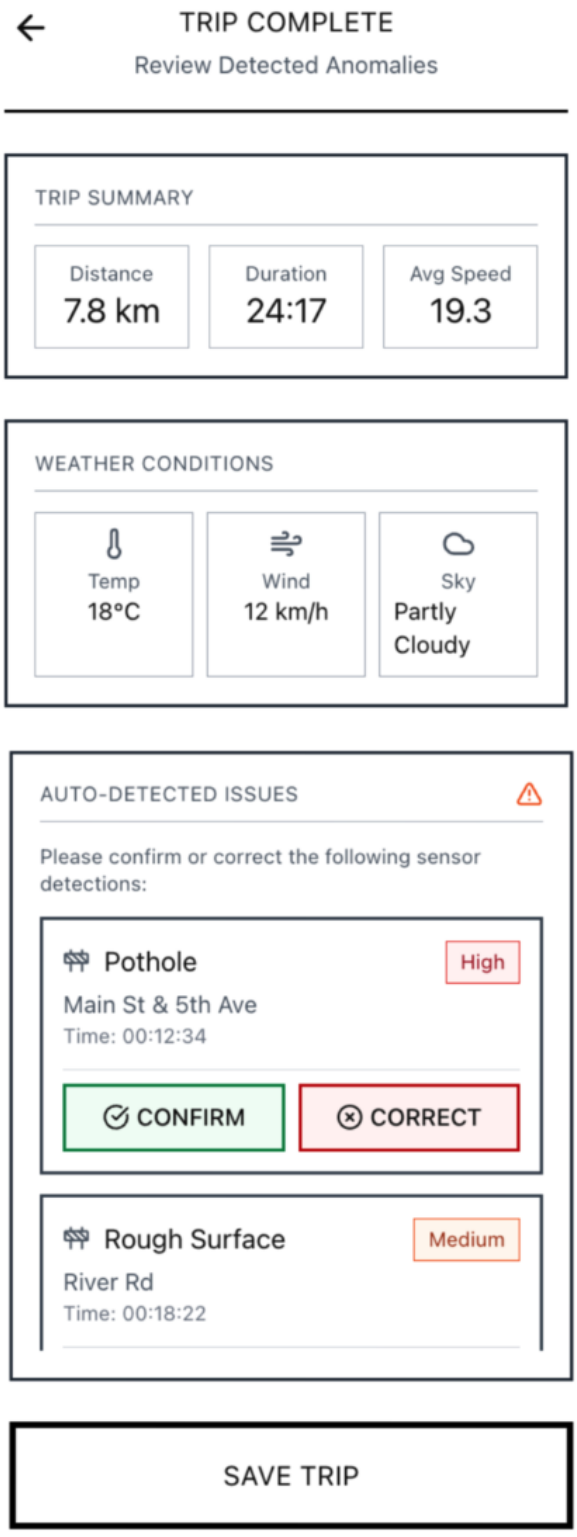


Figure 3.7: Summary and Anomaly Verification UI

Upon stopping the recording, the user is presented with the Trip Complete interface. The top section provides a "Trip Summary" containing the final statistics, total distance, duration, and average speed, recorded during the session. Consistent with the recording interface, the final meteorological conditions are also displayed to provide context to the performance metrics.

The core feature of this screen is the "Auto-Detected Issues" panel, which addresses the requirement for validating sensor data. As the system automatically detects anomalies (such as potholes or rough surfaces) using the device's accelerometer and gyroscope during the ride, they are listed here for user review. To mitigate potential false positives, the user is required to interact with each detected item by selecting "Confirm" to validate the issue or "Correct" to discard or modify it. Once all anomalies are reviewed, the "Save Trip" button finalizes the entry, storing the trip details and updating the global bike path inventory with the verified obstacles.

3.1.6. Manual Reporting Interface



Figure 3.8: Mockup of the BBP Manual Reporting UI

The Manual Report interface corresponds to the "Manual Mode" defined in the system requirements, enabling users to contribute to the bike path inventory without recording a full trip. This form is designed for the precise insertion of publishable information regarding specific road segments.

The core of the interface is a structured form requiring the user to identify the "Street Name," satisfying the requirement to specify the location of the path manually. Below

this, the "Status / Condition" dropdown allows the user to categorize the quality of the path (e.g., optimal, requires maintenance), while the "Severity" toggle (Low, Medium, High) provides granular detail on any reported obstacles. A text area for "Additional Notes" permits the inclusion of qualitative descriptions that may not fit into predefined categories.

To enhance usability, the interface includes a "Current GPS Location" feature, allowing users to quickly populate the location field using their device's current coordinates. Once the form is complete, the "Submit Report" button finalizes the entry, making the data available for the system to process and potentially merge with other user reports.

3.1.7. Trip Details and Publication

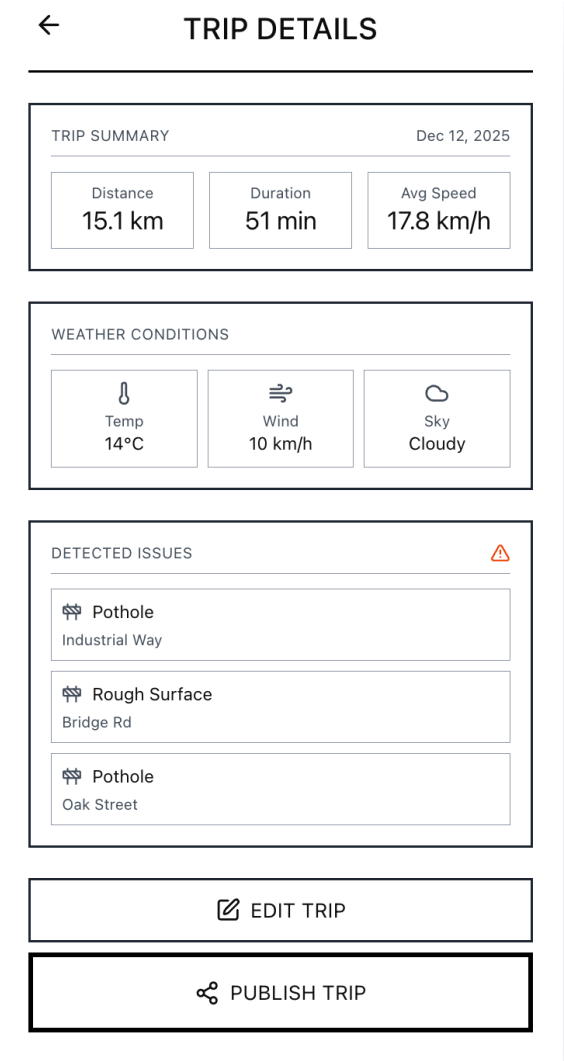


Figure 3.9: Mockup of the BBP Trip Details and Publication UI

Accessing a specific entry from the "Recent Trips" list on the homepage directs the user to the Trip Details interface. This view acts as a permanent record of the user's cycling activity, displaying the stored "Trip Summary" statistics (distance, duration, and average speed) alongside the historical "Weather Conditions" captured at the time of the ride.

Crucially, this interface serves as the final review stage before data sharing. The "Detected Issues" section lists the road anomalies (e.g., potholes, rough surfaces) that were confirmed during the recording phase or manually inserted. Users can review these details to ensure accuracy. The interface provides two primary actions: "Edit Trip," which allows for the retrospective modification of data (e.g., correcting a street name or adjusting issue severity), and "Publish Trip." Selecting "Publish Trip" changes the visibility of the data from private to public, thereby fulfilling the system requirement to make path status information available to the wider community for route calculation and merging.

3.1.8. Edit Trip Interface

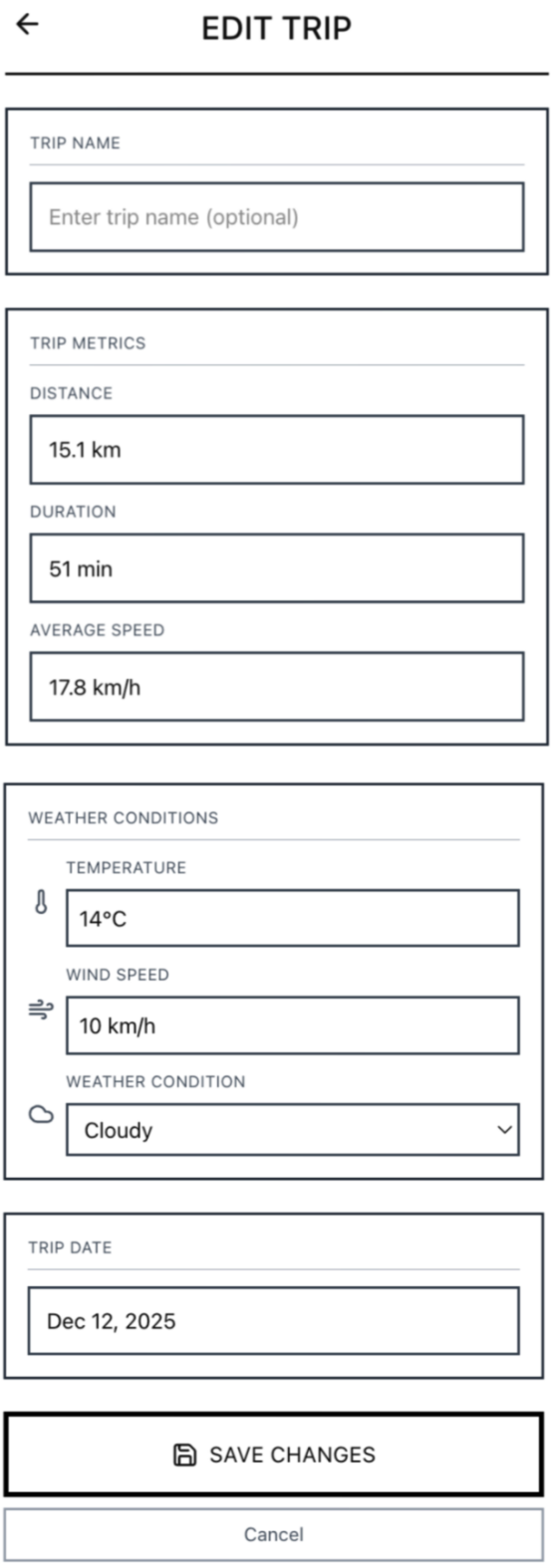


Figure 3.10: Mockup of the BBP Edit Trip UI

Initiated by selecting the "Edit Trip" action from the Trip Details view, this interface empowers the user to refine the data associated with a specific cycling activity. This functionality is essential for addressing the system requirement that allows users to correct information acquired by BBP before it is finalized or published.

The form is organized into modular sections. The "Trip Name" field allows users to assign a personal identifier to the route. Below this, the "Trip Metrics" section (Distance, Duration, Average Speed) and "Trip Date" are presented in editable fields, enabling the user to manually adjust values in case of GPS drift or sensor errors during the recording phase. Furthermore, the "Weather Conditions" section permits the modification of meteorological data such as temperature, wind speed, and general conditions (e.g., changing "Sunny" to "Cloudy"), ensuring that the context of the trip remains accurate even if the external weather service provided mismatched local data. The process is concluded by clicking "Save Changes," which updates the local record and prepares the validated data for potential publication.

3.1.9. Trip Publication Confirmation

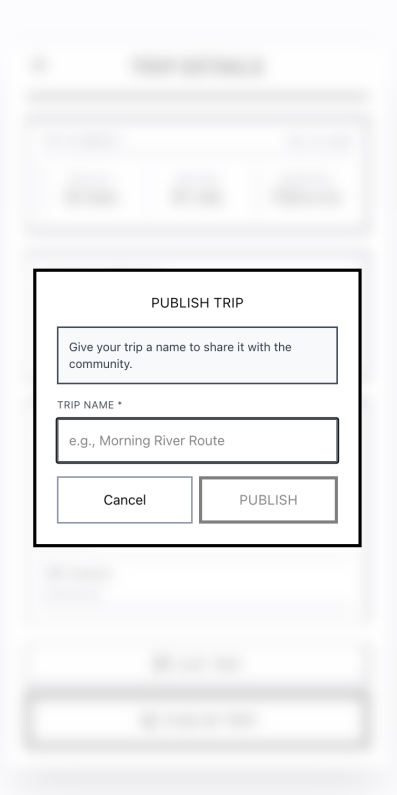


Figure 3.11: Mockup of the BBP Trip Publication Name Dialog

This modal interface appears as an overlay immediately after the user selects "Publish Trip" from the details view. It represents the final confirmation step required to transition the trip data from a private record to a shared community asset.

To ensure that published routes are identifiable and searchable by other users, the system mandates the assignment of a "Trip Name". The user is presented with a required text field to provide a descriptive title (e.g., "Morning River Route"). This step ensures that when the data is merged into the global bike path inventory, it carries a meaningful identifier. Clicking the primary "PUBLISH" button commits the transaction, making the path status and obstacle data available to the BBP community for future route planning and statistical aggregation.

3.1.10. Publication Confirmation

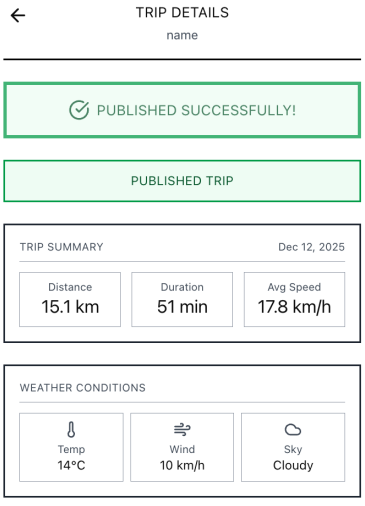


Figure 3.12: Success Feedback

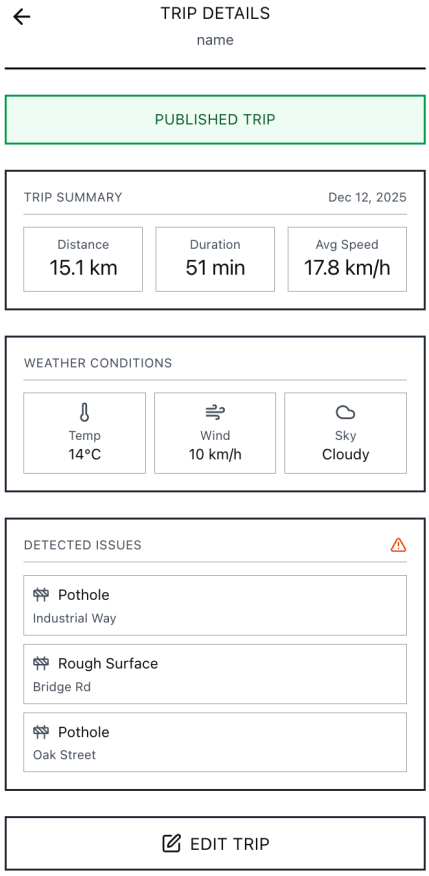


Figure 3.13: Permanent Published State

Figure 3.14: Trip Publication Workflow Results

Once the user confirms the trip name, the system executes the publication process. As shown in Figure 3.12, the interface provides immediate visual feedback via a transient "Published Successfully!" notification banner at the top of the screen. This confirms to the user that their data has been successfully transmitted to the server and merged with the global inventory.

Following this confirmation, the Trip Details view updates to reflect the new visibility status of the record. As depicted in Figure 3.13, a permanent green "Published Trip" indicator replaces the previous action buttons. This visual cue signifies that the path status and obstacle data are now public and accessible to the community. The user retains access to the trip statistics and weather data, but the record is now locked as a shared contribution.

3.1.11. Map and Navigation Interface

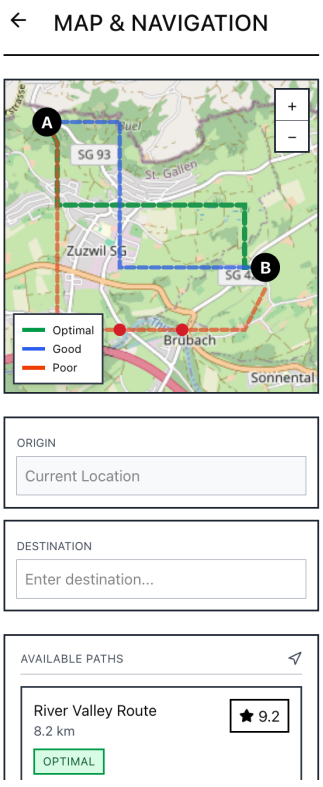


Figure 3.15: Map Visualization

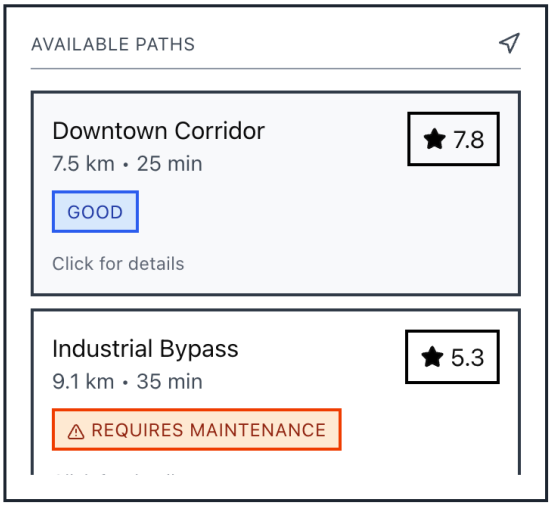


Figure 3.16: Path Selection List

Figure 3.17: BBP Map Navigation and Path Selection UI

Accessible via the "Search Trip" action on the homepage, the Map & Navigation interface serves the primary consumer goal of the BBP system: allowing users to find the best routes based on community data. The interface is driven by two primary input fields, "Origin" and "Destination," allowing users to query paths between specific points.

The central visual component (Figure 3.15) is the interactive map, which renders the available routes color-coded by quality (Green for "Optimal," Blue for "Good," and Orange for "Poor"). This visualization directly reflects the aggregated status information provided by other users.

As shown in Figure 3.16, the "Available Paths" section lists the specific route options ranked by a calculated "Score". For example, a path marked "Requires Maintenance" appears with a lower score (e.g., 5.3), alerting the user to potential issues. By clicking on a specific result in this list, the user can access expanded details, such as specific obstacle reports, and explicitly select the trip to commence navigation.

3.1.12. Guest Access Interface

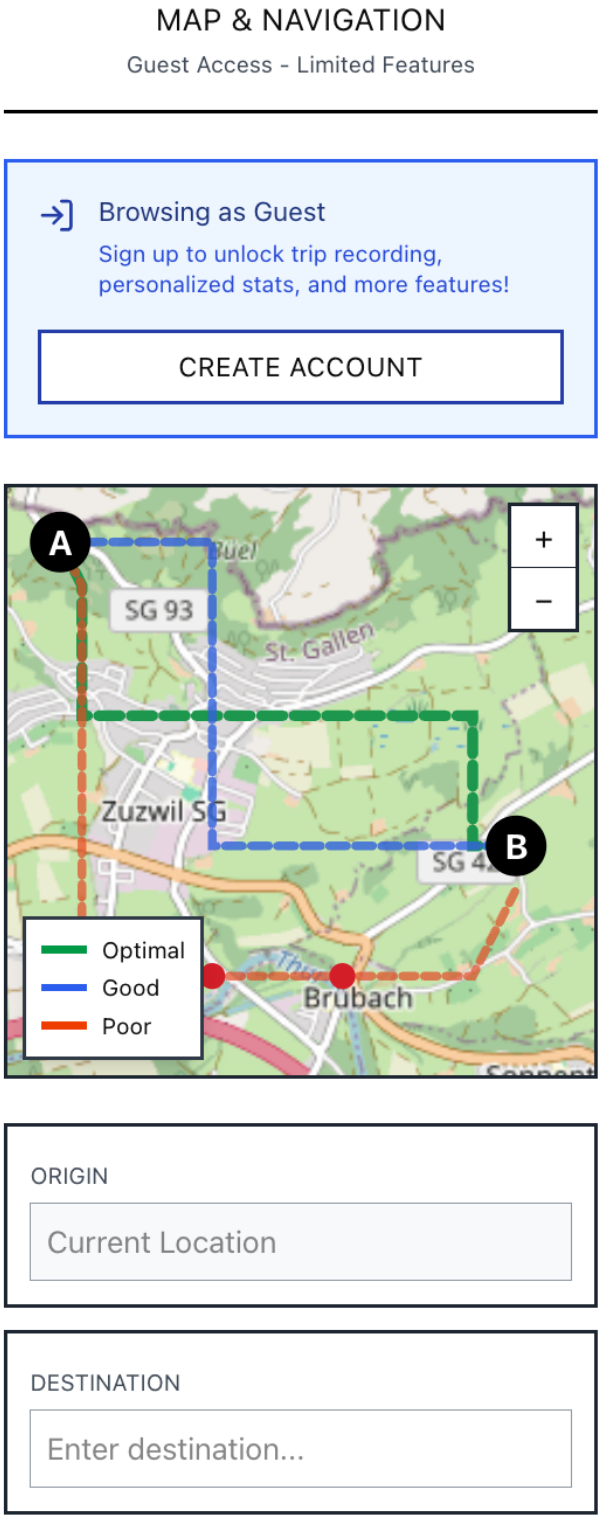


Figure 3.18: Mockup of the BBP Guest Access UI

The Guest Access interface represents the system's state when accessed by an unregistered user or a user who has chosen to "Continue as Guest" from the login screen. This design fulfills the critical requirement that "Any user, either registered or not, can take advantage of the information collected by BBP."

As shown in the mockup, the core "Map & Navigation" functionality remains fully accessible, allowing the user to input an Origin and Destination to search for bike paths. The visualization of path status (Optimal, Good, Poor) is identical to the registered view, ensuring safety information is public.

However, distinct UI elements clarify the limitations of this mode. A prominent banner at the top explicitly identifies the session as "Browsing as Guest" and includes a call-to-action "Create Account" button. This banner informs the user that advanced features, specifically trip recording, personalized statistics, and contributing data, are locked, thereby encouraging conversion to a registered account without blocking access to the essential utility of the application.

3.1.13. Conclusions

The user interface designs presented in this chapter demonstrate a comprehensive coverage of the functional requirements defined for the Best Bike Paths (BBP) system. The proposed visual hierarchy effectively distinguishes between the two primary user roles: enabling registered cyclists to actively contribute data through the *Automated* and *Manual* acquisition modes, while ensuring that unregistered guests have seamless access to the resulting community inventory, as mandated by the system specifications.

A key focus of the interface design is data integrity. The specific workflows for *Trip Completion* and *Trip Editing* introduce necessary human-in-the-loop verification steps, forcing users to validate sensor-detected anomalies before data is finalized. This design directly addresses the requirement to filter false positives from automated sources.

Finally, the transition from individual contribution to collective utility is visualized through the *Publication* and *Path Selection* screens. By explicitly separating private tracking from public sharing, the UI supports the system's goal of aggregating disparate user reports into a cohesive, scored, and reliable map for safe route planning.

4 | Requirements Traceability

This section establishes a mapping between the functional requirements specified in the Requirements Analysis and Specification Document (RASD) and the architectural components described in Chapter 2 of this Design Document. The mapping ensures that every system requirement is addressed by specific architectural modules, guaranteeing comprehensive coverage.

Users Account Manager Service

The **Users Account Manager Service** manages all logic related to user identity, ensuring strict isolation of Personally Identifiable Information (PII) as described in Section 2.2.1.

- **R01 (Registration)**: Handles user account creation and data persistence in the User Data store.
- **R02 (Login/Logout)**: Manages authentication logic and session token generation (JWT).

Mobile Application Components

The **Mobile Application** acts as the primary interface for data acquisition and user interaction. Its internal modules (Trip Manager, Sensor Manager, UI Controller, Map Manager) handle the recording loop and local data buffering.

- **R04 (Start recording)**: Trip Manager orchestrates the initialization of a new recording session.
- **R05 (Record coordinates)**: Sensor Manager interfaces with hardware to capture GPS samples.
- **R06 (TrackPoint generation)**: Trip Manager encapsulates raw sensor readings into TrackPoint objects.

- **R07 (Compute statistics)**: Trip Manager calculates real-time metrics (distance, speed) during the ride.
- **R08 (Set acquisition mode)**: UI Controller allows the user to select between Manual and Automatic modes.
- **R10 (Manual street entry)**: Map Manager captures user interactions with the map to define manual paths.
- **R13 (Confirm/Reject detected obstacles)**: UI Controller presents detected anomalies to the user for validation.
- **R21 (Display status and score)**: Map Manager renders color-coded paths based on their status and score.
- **R26 (Display street conditions)**: Map Manager visualizes specific street-level conditions and markers.
- **R27 (Browse trip history)**: UI Controller displays the list of past trips retrieved from the backend.
- **R33 (Sensor unavailability notification)**: Sensor Manager detects hardware failures and alerts the user.
- **R34 (GPS signal notification)**: Sensor Manager monitors location accuracy and notifies the user of poor signal.

Backend Trip Processing (Ingestion, Aggregation, Scoring)

The backend processing pipeline handles the high-volume writing operations, including data enrichment, merging, and quality scoring.

- **R09 (Associate weather)**: Trip Ingestion Service queries the WeatherAPI Wrapper to enrich incoming trip data.
- **R24 (Update aggregated score)**: Trip Scoring Service recalculates the Quality of Experience (QoE) score based on new data.
- **R29 (Mark data publishable)**: Trip Aggregation Service handles the publication request (`PUT /publish`) and triggers the merging process.

Report Server (Ingestion and Validation Services)

The **Report Server** is a dedicated component for handling crowd-sourced data, ensuring that discrete user reports do not interfere with real-time tracking.

- **R11 (Submit report)**: Report Ingestion Service accepts `ObstacleReportDTO` payloads from the client.
- **R12 (Pending status)**: Report Ingestion Service initializes reports with a PENDING status upon receipt.
- **R14 (Update report status)**: Report Validation Service updates the status to CONFIRMED or REJECTED after validation.
- **R15 (Associate/Create obstacle)**: Report Validation Service links valid reports to persistent Obstacle entities.
- **R16 (Obstacle publishability)**: Report Ingestion Service persists the user's visibility preference for the report.
- **R17 (Store obstacle data)**: Report Server persists validated obstacles in the Obstacles & Reports Data store.
- **R18 (Associate obstacle with street)**: Report Validation Service performs map-matching to link obstacles to street segments.
- **R19 (Update visibility)**: Report Server enforces visibility rules based on the publishability flag.
- **R22 (Submit path status report)**: Report Ingestion Service receives qualitative feedback on path conditions.
- **R23 (Path report publishability)**: Report Ingestion Service stores the publishability setting for path status reports.
- **R35 (Prevent invalid reports)**: Report Validation Service filters out duplicates and spatially inconsistent data.

Trip Search Server

The **Trip Search Server** is a read-optimized component implementing the query side of the CQRS pattern. It is solely responsible for retrieving and filtering data for the user.

- **R03 (Unregistered access)**: Handles public API requests without requiring a

user session token.

- **R20 (Represent path sequences)**: Returns `BikePathDTO` objects containing street sequences and geometry.
- **R25 (Browse paths)**: Executes spatial queries to find and return bike paths matching search criteria.
- **R28 (View trip details)**: Retrieves comprehensive trip data, including weather and obstacles, for detailed viewing.
- **R30 (Hide private reports)**: Filters out non-publishable data from query results before returning them to the client.

External Service Adapters

These wrapper components abstract the interaction with third-party APIs.

- **R31 (Retrieve weather data)**: WeatherAPI Wrapper fetches meteorological data from external providers.
- **R32 (Retrieve map data)**: MapAPI Wrapper interfaces with mapping services (e.g., OpenStreetMap) for routing and geocoding.

5 | Implementation, Integration and Testing

5.1. Overview and Implementation Plan

This chapter outlines the implementation approach, integration strategy, and testing methodology for the Best Bike Paths (BBP) system. The development will follow a **Bottom-Up approach**, starting with lower-level infrastructure modules and gradually integrating them to form higher-level business logic components.

This methodology ensures that foundational modules—such as data access layers, external service adapters, and raw sensor handlers—are thoroughly validated before being incorporated into more complex modules like the Trip Ingestion Service or the Aggregation Engine. Each module will be tested using dedicated **Test Drivers** that simulate the behavior of higher-level components. For instance, a driver might simulate GPS coordinate streams to test the tracking module without requiring physical device movement.

The Bottom-Up strategy facilitates incremental integration and enables early detection of defects in isolated components. It also supports parallel development efforts; different teams can simultaneously work on mobile sensor acquisition logic and backend processing algorithms, accelerating the overall timeline.

On the client side, the Mobile Application comprises the dedicated managers defined in Section 2.2 of the Architectural Design: the **Sensor Manager**, **Local Database Manager** (for offline buffering), **Network Manager** (for connectivity handling), and the **UI Controller**. These components handle sensor data acquisition, temporary persistence during offline usage, network recovery, and user interface rendering. They are developed concurrently with backend services and integrated during the final phase.

5.2. Features Identification

The BBP system offers functionality to different user categories and includes background system processes. Features are categorized as follows:

RU Features (Registered Users): Enable cyclists to manage accounts, record trips, and contribute to the path inventory.

- **[RU.1] Registration and Login**: Create an account via email, verify identity, and securely authenticate.
- **[RU.2] Profile Management**: Update personal information and application preferences.
- **[RU.3] Manual Trip Creation**: Define a route by selecting streets on a map without sensor tracking.
- **[RU.4] Automatic Trip Recording**: Record trips in real-time using device sensors (GPS, accelerometer).
- **[RU.5] Obstacle Reporting**: Report obstacles (e.g., potholes) encountered during or after a trip.
- **[RU.6] Path Status Rating**: Provide qualitative feedback on the condition of a specific street or bike path.
- **[RU.7] Validation of Detections**: Confirm or reject obstacles automatically detected by the system.
- **[RU.8] Publishability Management**: Control whether trips and reports are private or publicly visible.
- **[RU.9] Dashboard and History**: View cycling statistics, past trips, and performance metrics.

UU Features (Unregistered Users): Available to all users without authentication for exploring community data.

- **[UU.1] Map Browsing**: Browse interactive maps with bike paths overlaid on street networks.
- **[UU.2] Route Search**: Search for bike paths between origin and destination points.

- **[UU.3] Filtering and Sorting**: Filter search results by path scores or status (e.g., only "Optimal" paths).
- **[UU.4] Obstacle Visualization**: View confirmed obstacles and warnings along computed routes.

SYS Features (System Processes): Background operations that maintain data consistency and enrich user experience.

- **[SYS.1] Path Score Calculation**: Compute quality scores for paths based on aggregated reports and data freshness.
- **[SYS.2] Status Aggregation**: Merge conflicting reports using majority-voting logic to determine authoritative path status.
- **[SYS.3] Anomaly Detection**: Process raw sensor data to infer potential anomalies during trips.
- **[SYS.4] External Data Enrichment**: Integrate external data, such as weather conditions for trip context.

5.2.1. Mapping Features to Architectural Components

The following mapping clarifies which architectural components (defined in Chapter 2) implement each feature group:

- **[F1] Account Management Features**: Implemented by the *Users Account Manager Service* (backend) and client-side authentication logic.
  - **[RU.1] Registration and Login**: Handle credentials, session tokens, and security checks.
  - **[RU.2] Profile Management**: Update user personal details in the database.
- **[F2] Trip Management Features**: Implemented by the *Trip Manager* (mobile) and *Trip Ingestion Service* (backend).
  - **[RU.3] Manual Trip Creation**: Interface for selecting street segments and saving as a trip entity.
  - **[RU.4] Automatic Trip Recording**: Tracking logic storing GPS streams as TrackPoint sequences.
  - **[RU.9] Dashboard and History**: Data retrieval logic for displaying historical trip data.

- **[F3] Sensor & Detection Features**: Implemented by the *Sensor Manager* (mobile) and *Trip Validation Service* (backend).
  - **[SYS.3] Anomaly Detection**: Algorithms analyzing accelerometer variances to flag potential potholes.
  - **[RU.7] Validation of Detections**: Notification system prompting users to verify inferred data.
- **[F4] Reporting Features**: Implemented by the *Report Server* (backend) and client-side reporting interfaces.
  - **[RU.5] Obstacle Reporting**: Creation of ObstacleReport entities linked to specific coordinates.
  - **[RU.6] Path Status Rating**: Logic for submitting PathStatusReports for specific paths.
  - **[RU.8] Publishability Management**: Privacy filters checking the isPublishable flag before serving data.
- **[F5] Aggregation & Scoring Features**: Implemented by the *Trip Aggregation Service* and *Trip Scoring Service* (backend).
  - **[SYS.1] Path Score Calculation**: Mathematical formulas ranking paths based on quality and safety.
  - **[SYS.2] Status Aggregation**: Logic querying active reports and applying majority rule to update path status.
- **[F6] Map & Search Features**: Implemented by the *Map Manager* (mobile) and *Trip Search Server* (backend).
  - **[UU.1] Map Browsing**: Integration with external map providers (e.g., OpenStreetMap) to render layers.
  - **[UU.2] Route Search**: Geospatial queries finding paths between two GeoPoints.
  - **[UU.3] Filtering and Sorting**: Sorting algorithms applied to retrieved path lists.
  - **[UU.4] Obstacle Visualization**: Frontend components displaying markers for confirmed obstacles.

- **[F7] External Services Features**: Implemented by the *WeatherAPI Wrapper* and *MapAPI Wrapper* (backend adapters).
  - **[SYS.4] External Data Enrichment**: Connector to external weather services for fetching temperature and conditions.

5.3. Integration Strategy

Integration and testing commence once the DBMS and backend infrastructure are configured. The Bottom-Up approach (Section 5.1) guides the process, beginning with foundational modules and progressively adding higher-level components.

5.3.1. Step 0: Data Access & External Connectors

The initial phase implements the **Data Access Layer (DAL)** and **External Service Adapters**. The DAL manages database connections and returns entity objects used by all services. Simultaneously, adapters for the Weather Service and Map Provider ensure reliable external data retrieval. These modules are tested using drivers simulating calls from business logic components.

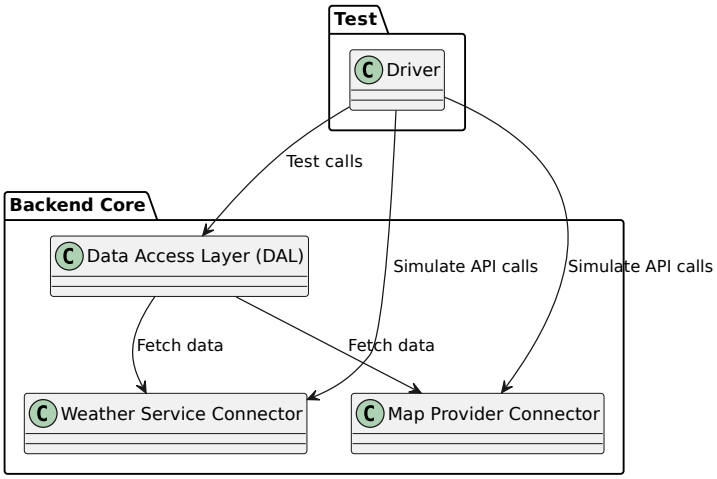


Figure 5.1: Integration Strategy - Step 0: Data Access & Connectors

5.3.2. Step 1: User Management Foundation

The **Users Account Manager Service** (implementing [F1] features) is integrated next. This service handles registration, authentication, and profile management, establishing the security foundation required for most system functionalities.

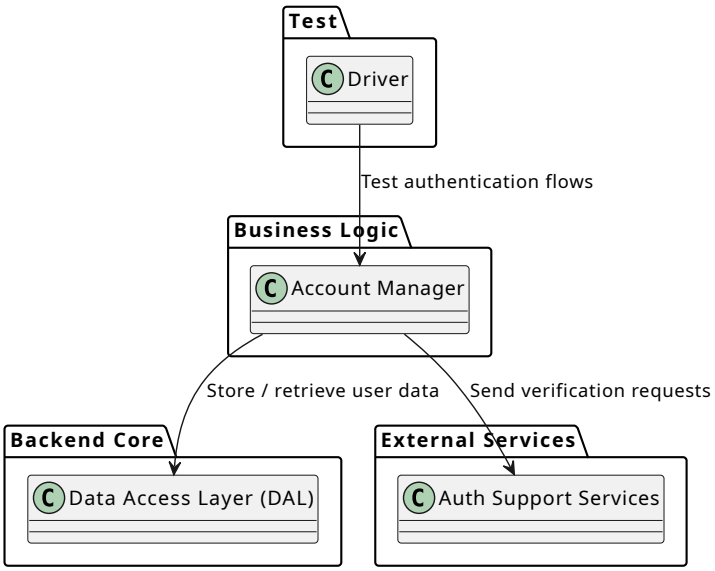


Figure 5.2: Integration Strategy - Step 1: Users Account Manager Service

5.3.3. Step 2: Core Processing Algorithms

The **Sensor & Detection Handler** ([F3]) and **Aggregation & Scoring Engine** ([F5]) are implemented concurrently. These modules contain the system's core mathematical logic—anomaly detection and path scoring—which operate independently of user session state, ensuring algorithm robustness before UI integration.

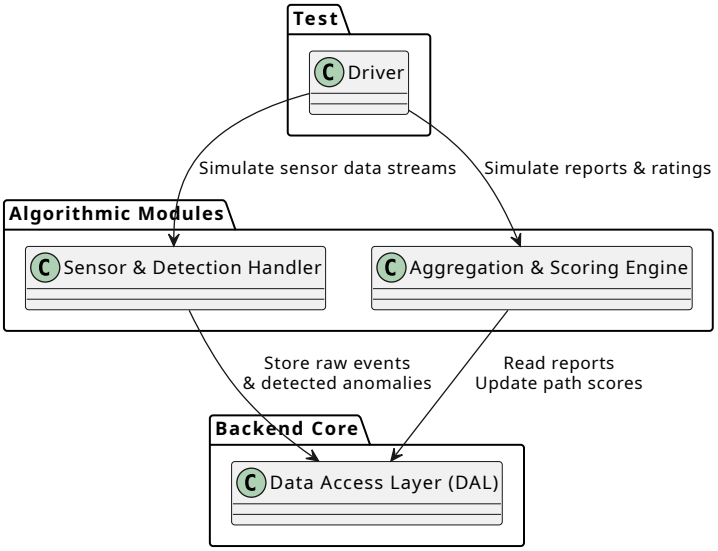


Figure 5.3: Integration Strategy - Step 2: Core Algorithms

5.3.4. Step 3: Trip and Reporting Orchestration

The **Trip Ingestion Service** ([F2]) and **Report Server** ([F4]) are integrated as central orchestrators. The Trip Ingestion Service coordinates with the Users Account Manager (for user linkage), Sensor Handler (for data recording), and external adapters (for weather enrichment). The Report Server processes user submissions and interacts with the Aggregation Engine to update path statuses.

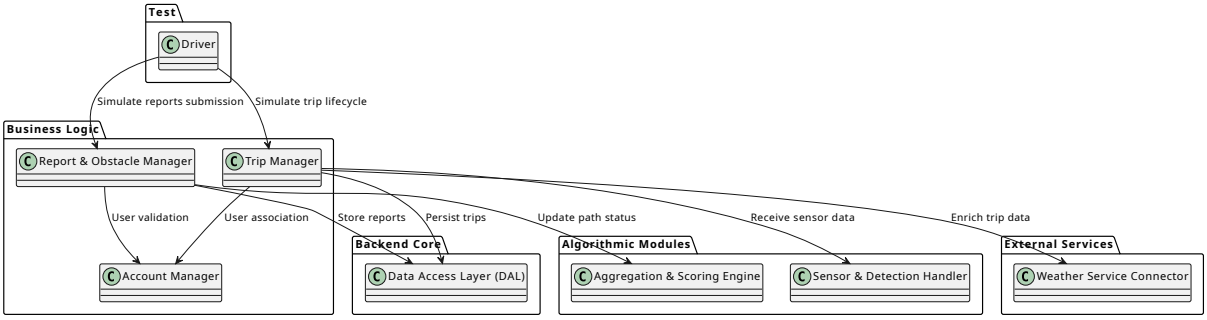


Figure 5.4: Integration Strategy - Step 3: Trip Ingestion and Report Services

5.3.5. Step 4: Search and Visualization Layer

The **Trip Search Server** ([F6]) and **Dashboard Manager** ([F2/RU.9]) form the consumption layer. The Trip Search Server executes optimized queries over aggregated data, while the Dashboard Manager retrieves historical data for statistics visualization.

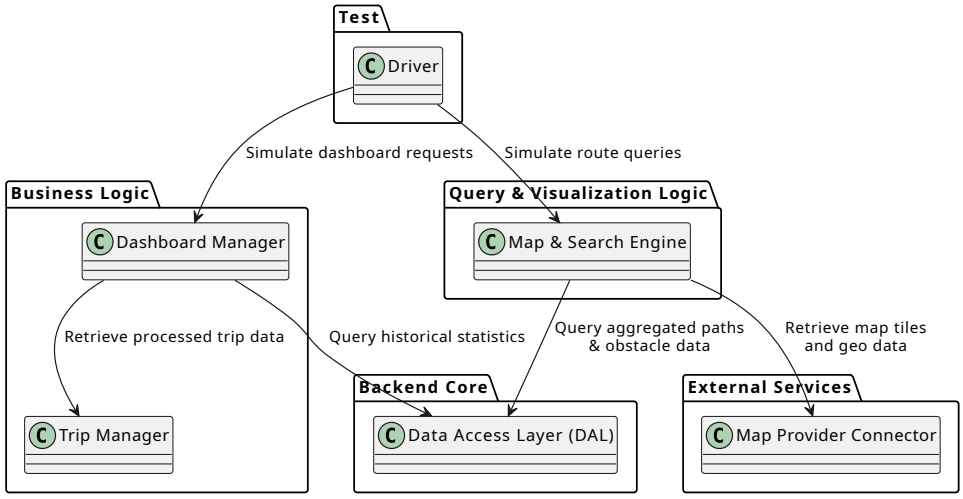


Figure 5.5: Integration Strategy - Step 4: Search and Dashboard

5.3.6. Step 5: Client Application Integration

The final phase integrates client applications (Mobile App) with backend APIs. This validates end-to-end communication, JSON response handling, and interaction among client-side managers: Sensor Manager, Local Database Manager, Network Manager, and UI Controller.

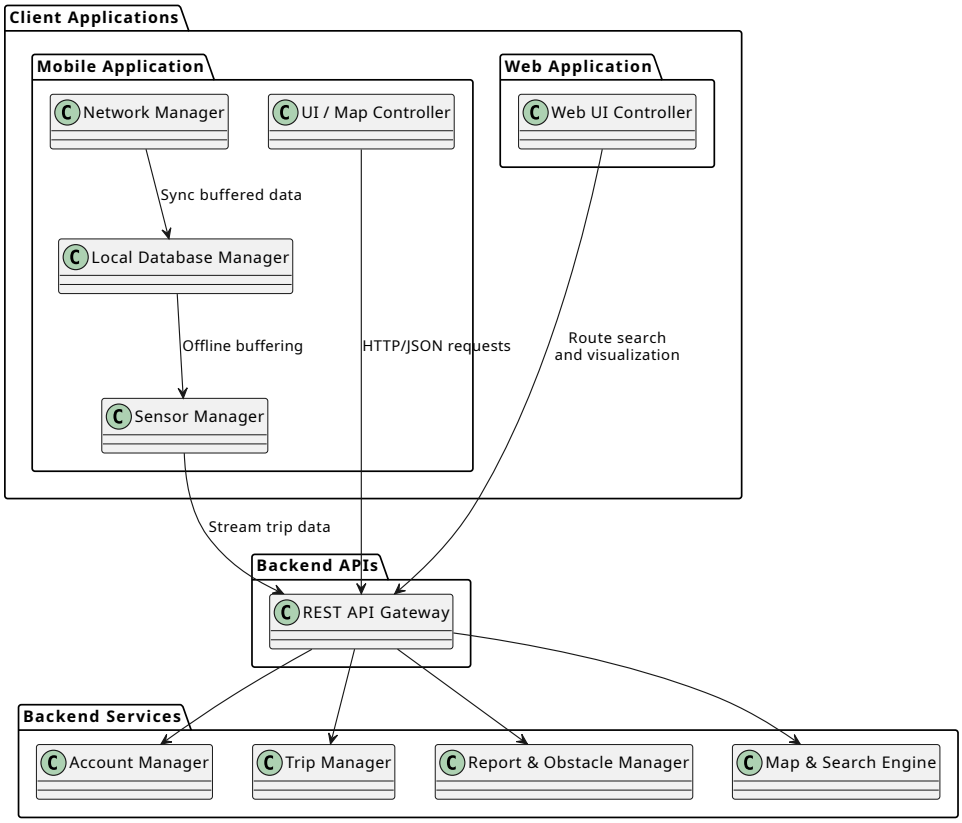


Figure 5.6: Integration Strategy - Step 5: Client Integration

These client-side components correspond precisely to those defined in Section 2.2 (Architectural Design).

5.4. System Testing Strategy

Following component-level testing, the system undergoes comprehensive evaluation using a **Black-Box strategy**, assessing functionality from the user's perspective without internal implementation knowledge.

The testing regimen includes:

1. **Functional Testing**: Validates that critical workflows (e.g., [RU.4] Automatic Trip

Recording, [UU.2] Route Search) operate as specified in the RASD requirements.

2. **Performance Testing**: Confirms the system handles expected loads of 50,000 active users, ensuring backend ingestion of high-frequency sensor data without latency and map rendering within 2 seconds.

3. **Security Testing**: Verifies GDPR compliance for location data, ensures private trip isolation, and validates API endpoint resilience against SQL injection, token hijacking, and unauthorized access.

4. **Failure Testing**: Simulates adverse conditions (GPS signal loss, network interruptions) to confirm graceful recovery and data preservation, addressing Constraint C05 (intermittent connectivity).

5. **Endurance Testing**: Assesses system stability under prolonged use (e.g., 4-hour trip recording), checking for battery efficiency, resource leaks, and backend stability.

6. **User Acceptance Testing (UAT)**: Real cyclists execute realistic tasks (riding varied terrains, reporting obstacles) to validate that the system meets practical needs and delivers satisfactory Quality of Experience (QoE).

6 Effort Spent

Group Member	Effort Spent	
Giuseppe	Introduction	*1 h*
	Architectural Design	*3 h*
	User Interface Design	*9 h*
	Requirements Traceability	*2 h*
	Implementation, Integration and Testing	*1 h*
	Reasoning	*3.5 h*
Lorenzo	Introduction	*1 h*
	Architectural Design	*12 h*
	User Interface Design	*2 h*
	Requirements Traceability	*0.5 h*
	Implementation, Integration and Testing	*1 h*
	Reasoning	*3.5 h*
Giacomo	Introduction	*1.5 h*
	Architectural Design	*3 h*
	User Interface Design	*1.5 h*
	Requirements Traceability	*0.5 h*
	Implementation, Integration and Testing	*9.5 h*
	Reasoning	*3.5 h*

Table 6.1: Effort Spent by Each Member of the Group Table

7 References

7.1. References

- ISO/IEC/IEEE 29148:2018 - Systems and software engineering - Life cycle processes - Requirements engineering.
- The Requirement Engineering and Design Project specification document A.Y. 2025-2026.
- Lecture slides and notes from Prof. Di Nitto's course
- `https://github.com/MarcoApollonio02/ApollonioBossiChiroli`

7.2. Used Tools

- **GitHub** - Used for project versioning and sharing.
- LaTeX and **Visual Studio Code** - Used as editor for writing the document.
- **draw.io** - Used for diagrams' design.
- **PlantUML** - Used for diagrams' design.
- **Figma** - Used for the mockups' design.